SOCIAL NETWORKS

NPTEL Online Course | Prof. Sudarshan Iyengar, IIT Ropar

Week 6 - Lecture Transcripts

Lectures 75-86 | 12 lectures | 82 pages (source transcript pages 696-776)

INDEX

LECTURE	TOPIC	PAGE IN THIS FILE
L75	The Web Graph	2
L76	Collecting the Web Graph	8
L77	Equal Coin Distribution	17
L78	Random Coin Dropping	23
L79	Google Page Ranking Using Web Graph	29
L80	Implementing PageRank Using Points Distribution Method-1	32
L81	Implementing PageRank Using Points Distribution Method-2	37
L82	Implementing PageRank Using Points Distribution Method-3	48
L83	Implementing PageRank Using Points Distribution Method-4	60
L84	Implementing PageRank Using Random Walk Method-1	67
L85	Implementing PageRank Using Random Walk Method-2	70
L86	DegreeRank versus PageRank	78

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Link Analysis
Lecture – 75
The Web Graph

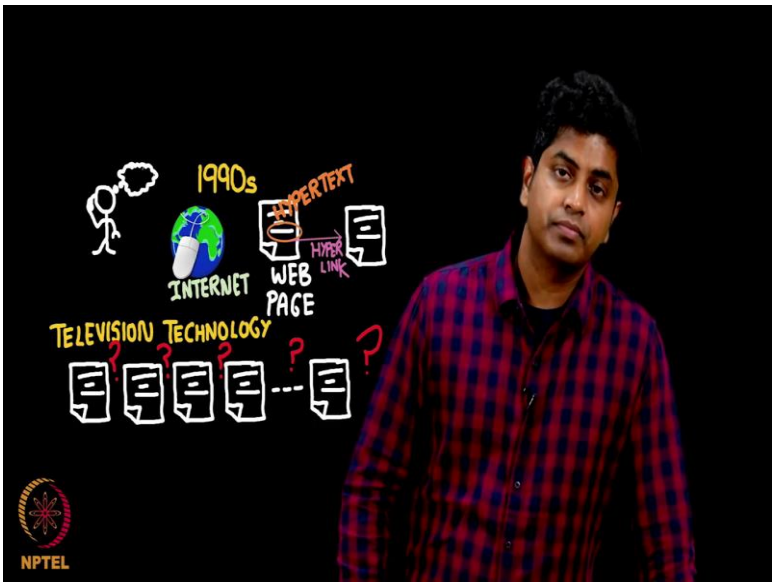
Let us down switch directions and talk about something that I spoke in at length in the introductory video.

(Refer Slide Time: 00:19)



It is about the origin of Google as an industry and the algorithm that they used to search the web. So, I will now call upon you all to imagine the following story.

(Refer Slide Time: 00:29)



Please note you really have to imagine ~~otherwise,~~otherwise; you cannot get the gist what I am going to say. Please imagine that me and you are in the 1990s and I have this word with you that, see there is something called the internet which is fast developing. There is something called a web page where people put some information and there is something called hyper link, a hypertext where in the page you can link to another page. You click here, it just goes there right that is what you mean by hyperlink. By a link you mean something that links to somewhere else, but hyperlink means something that not just links hyper means what more than a link hyperlink is more than a link. Hyperlink means it just does not link you to a new place; it even takes you to that place.

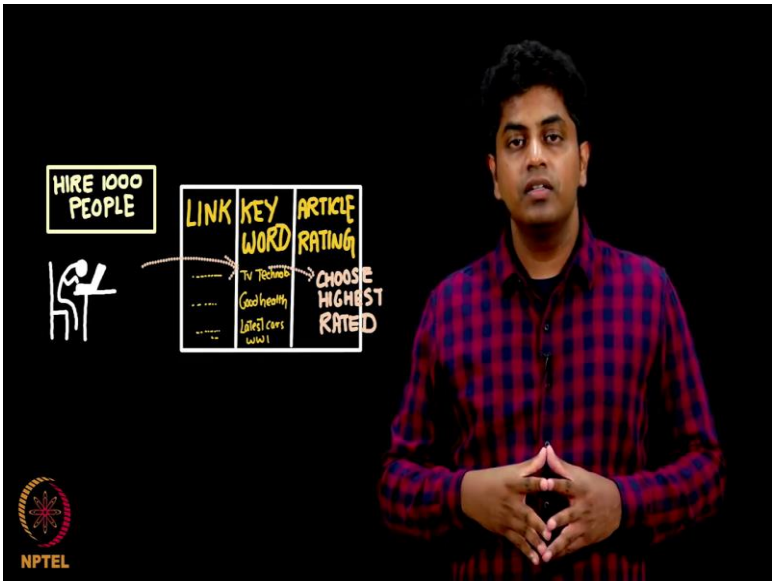
So, a ~~hyperlinks~~hyperlink was the invention of hypertext was very big leap in computer science. So, there are pages with hyperlinks to other pages and now me and you are thinking of a start up idea. In the 1990 where we see that internet is booming with a lot of web pages which are linked ~~one page~~one-page links to the other page through a hyper link and we all start thinking. Now there are so many pages here thousands and thousands of pages. Today actually its billions of ~~billions~~billions of pages, billions of pages, but I am talking about 1990; me and you are having coffee table conversation in nineteen ninety.

Look there are so many web pages. How do you know what to search for? How do you know which page has the right information? Assume there are so many pages, web pages

talking about television technology, the television technology. Which page is a authoritative source for me to refer? If there are 100 pages which talk about 100 different web pages written by 100 different technicians, technology experts. Which is the best possible webpage? Now I see I repeat we are in 1990s, I see that this probably becomes a problem is going to become a problem very soon. Now that there are few pages people can sort of go through all that page and decide which one is good and then read it, but very soon the web pages will be populated with hundreds of thousands of pages. How on earth will people know what to look for where right? What is the immediate idea, you think we both will get over this coffee table discussion in 1990?

I for a reason would give this dumb idea that fine we will hire some thousand people. Assume we had we had so much money to pay thousand people; I will say we will hire thousand people and ask them to go through each and every page and maintain a sheet which has the following entries following columns.

(Refer Slide Time: 03:39)



First column will be the link second column will be the keywords which tells me what ~~is this link~~this link is all about. This is the link I will say it talks about television technology. There is another link, this talks about health tips for good health; another link that talks about latest cars, another link that talks about World War-I so on right. First column will have the link, second column will have ~~this keywords~~these keywords what I what is the link all about. The third column will be, I will read by me I mean ~~this~~

~~thousand employees~~these thousand employees I am going to hire, each one of them will read the link and give the article rating. Or this article on World War-I is 8 on 10 rating, 8 on 10 like how the give rating for movies. Another person sees another link on World War-I and says this is not so good; it is 3 on 10 so on and so on and so on. These thousand employees in my start up industry will go through every single possible page ever created, ~~somehow~~somehow, we get access to all the pages let us say and we rate all the pages and we also put keywords.

So, that whenever someone searches for a keyword television technology, then it will come to this database of all the entries that my thousand people whom I have hired have populated. It will look at the highest rated page which has the keyword television technology and display that to him. Maybe it will sort it in the order of rating, best rated link first next best second, third, fourth descending order. It will show it you wonderful idea, everybody will come to my page to search for information on what is the ~~so called~~so-called internet with so many pages. Very soon I will become a millionaire because everybody will be coming to my page, my page will become popular. And once you have a popular portal, you can make a lot of money through ads or whatever right.

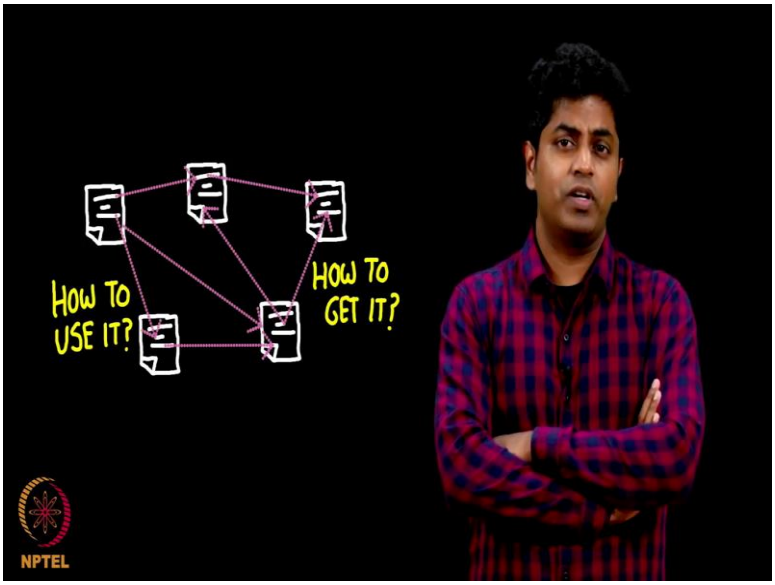
So, here is my idea in 1990 let us say, but then nobody ever dreamt that there could be a much ~~much~~ better way of doing this. I am going to explain next: what is the much ~~much~~ better way which paved its way for the birth of as I said a ~~450 billion dollar~~450-billion-dollar industry called Google; obviously, my start up idea will not scale up. Thousand people can do how much of work? The number of pages that gets populated on the internet is in terms of millions. So, very soon I will run out of man hours I am I will be need of more man hours and less employees; obviously, thousand is not enough. You should think of some other way of doing it right. What is this way of doing? It is ~~very~~ simple I will tell you the simple idea which is behind searching and again I will try explaining it bit by bit with analysis examples and questions; very simple.

(Refer Slide Time: 07:23)



Larry Page and ~~Surgey~~Sergey Brin, they came out with his idea of maintaining what is called a web graph.

(Refer Slide Time: 07:35)



By a web graph you mean take a page a webpage, call it a node. Take all the web pages, each web page is represented by a node. You wire a link between this page and this page namely this node and that corresponding node by a link. If this page has a hyperlink to that page; for example, the same example I have been giving you people. This is my homepage and I link the homepage of my friend on my homepage. Then my homepage is

a node, my friends homepage is a node; there is an edge from my home page to my friends homepage.

Please note this is a directed graph, why? Me linking to Prime Minister's homepage does not imply that the Prime Minister's homepage also links to mine. It is completely one sided correct. I like her, she does not mean she likes me. It is a very asymmetric relation. So, this is what is called a web graph where pages are denoted by nodes and a link represents a page pointing to another page through a hyperlink. The point is if you collect this web graph that is enough it does the trick of having thousand people or more. First question, how do I collect the web graph, what is the use of this web graph, how is this web graph going to solve the problem that I was trying to resolve with some thousand employees by hiring thousand employees right.

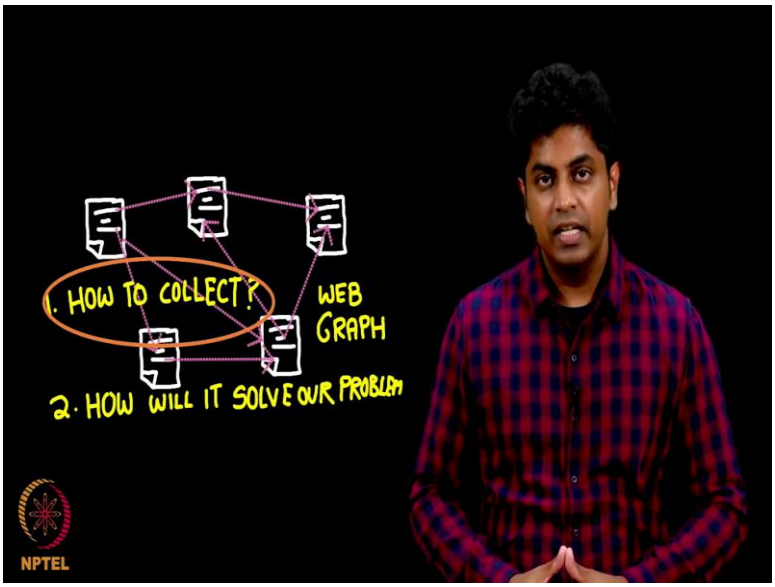
I have thousand employees in my organisation and now I am saying you can replace these thousand employees or even more by simply making node of the web graph. How do we get a web graph? What is the web graph? Think about it. It is a graph containing so many nodes and so many links. How do you even now the in some nook corner of the world, there will be a web page point into some other web page I am saying that you should have the all this information. How do you get this information? Do you have any idea how one can get this information? Any rough idea? Think about it.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Link Analysis
Lecture - 76
Collecting the Web Graph

So, let us be organised. We are only asking these two questions. Let us answer them one at a time.

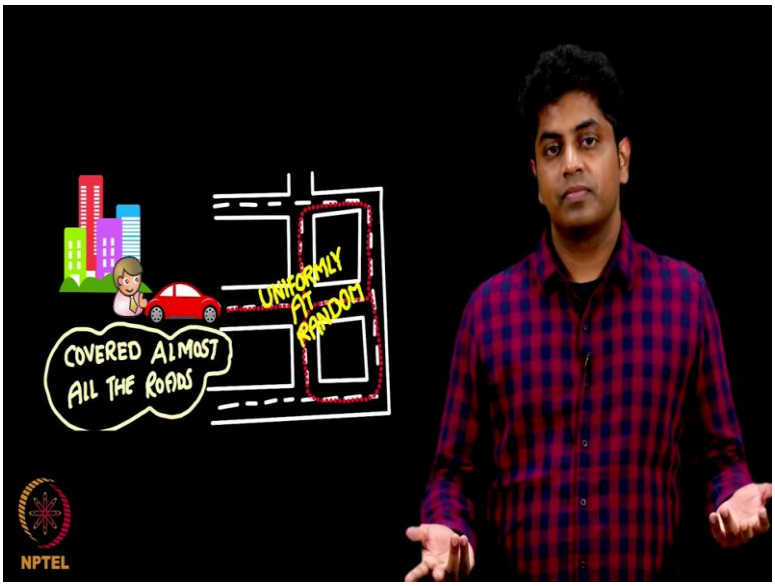
(Refer Slide Time: 00:15)



There is this question of web graph which will solve our problem; our problem of searching the web looks like unrelated you see. A graph of who points to whom which page points to what, has nothing to do with your search problem. So, first question is how we collect this web graph? I am saying this web graph will solve the problem. How will it solve the problem? Two things: firstly, how do you collect this web graph; secondly, how do you think this web graph will solve your problem.

Let us go step by step. So, first question. How do we collect this web graph? Here is a technique to collect the web graph. You should imagine a small situation.

(Refer Slide Time: 01:03)

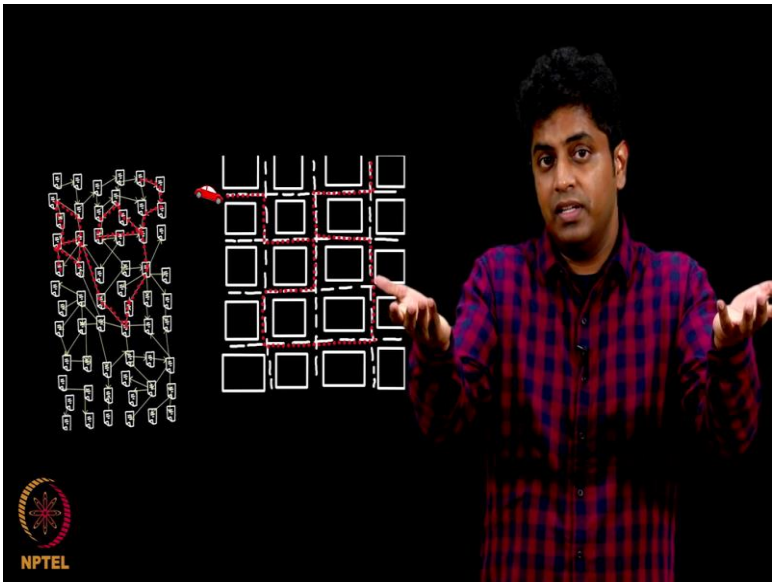


Imagine a city that you have never visited. And what I will do is I will put you to this new city; I will put you to this new city. Assume there is there are no vehicles in the city at all, you are the only one with a nice vehicle. I have given you a driver, a car road are empty; I repeat a new city you do not know of. Let us say you are not given a driver you drive the car, because if you are given a driver will know the localities. So, you are in a new city you do not know the localities, you are given the car; there is no traffic at all. What you do is you fix a camera on your car and just keep taking random turns here and there and keep going and going and going for the next one week. Assume you are given enough rest in between; you are given enough food in between; all you need to do is to simply take random turns and keep going.

Obviously you will not go a left and a right and a right and; obviously, you will not take a right and a right and a right and a right; you will come back to the same place, I said randomly. In every junction you toss a coin, you will take a decision whether you want to move towards your left or towards your right or towards straight. Or if there is a junction with 5 roads going from there, you roll a dice and you decide which side to go uniformly a track. You keep doing this for let us say, 1 week or more be even more may be 1 month. At the end of 1 month do you think there will be a road that you would not have explored?

May be yes, you would not have intuitively you feel you would not have. But do you think, you would have explored most of the roads in the city? Intuitively yes, you feel you would have if you take a car and keep going randomly wherever is an intersection, you break the tie by choosing a road uniformly at random and then move in that road and keep doing this for about a month's time in the city. You would have covered almost all roads right almost all roads you would have covered. Now what does this tell you? Does it tell you anything about the question that I just now asked?

(Refer Slide Time: 03:31)



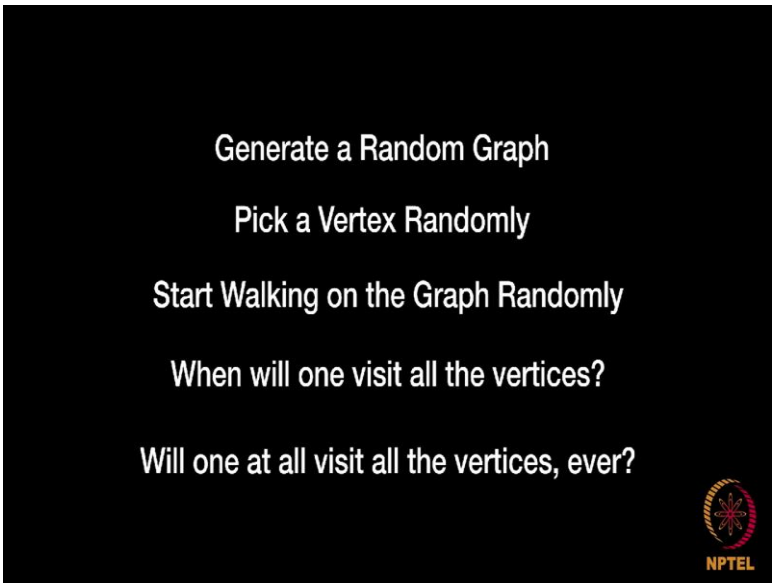
Is there a way in which you can explore this entire web graph and capture it? I gave you an example and you have this question in your hand, how do you get the web graph. The car, new city, taking random turns example; do you think you can link these two things? And do you think a question here has a solution which I stated in the form of an example?

(Refer Slide Time: 04:01)



So let us now do a quick experiment. I am curious to see if one takes a random walk on a given graph, how much time will it take for one to reach the entire graph by that I mean all vertices in the graph?

(Refer Slide Time: 04:19)

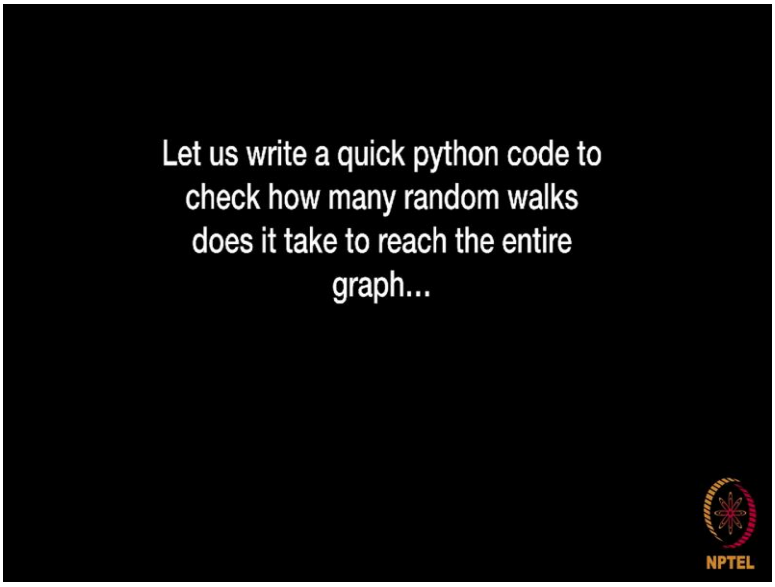


So, let us go slowly. I now need to generate a random graph on some n vertices with some probability p and then I pick a vertex randomly call it the start vertex. I will start walking on the graph randomly from this start vertex and then my question is when one will visit all the vertices in the graph. Will one at all visit all the vertices, ever? Pause for

a minute and think about this question and the what you think is your answer for this question.

Do you think it will take a long time or sometimes it is never possible to reach all the nodes by just taking a random walk? What do you mean by this? In a city, if you take a cab and then keep visiting places uniformly at random by tossing a coin and deciding whether to take a left turn or a right turn, how much time will it take for you to explore the entire city?

(Refer Slide Time: 05:21)



Let us now write a piece of code a quick python code to check how many random walks does it take to reach the entire graph. By reach the entire graph I mean reach all the vertices of the given graph.

(Refer Slide Time: 05:41)

```
import random
import matplotlib.pyplot as plt
import networkx as nx
import numpy

def walk(n,p):
    start=random.randint(0,n-1)
    G=nx.erdos_renyi_graph(n,p)
    S=set([])
    v=start
    count=0
    while(len(S)<n):
        Nbr=nx.neighbors(G,v)
        v=random.choice(Nbr)
        S.add(v)
        count=count+1
    return count

l=[]
for i in range(20,300):
    z=[]
    for j in range(10):
        z.append(walk(i,0.3))
    l.append(numpy.average(z))
    print i,"--->",numpy.average(z)
plt.plot(l)
plt.show()

~
~
"checkrandom.py" 28L, 522C written
```



Here is a piece of python code that have written. I am sure you people are, I am sure you people all of you are very familiar with python right now. You know how to write a piece of code. So, I have not gone a line by line. I have finished this code and you can probably take a pause and then take a look at this code and then judge what the code is doing. So, as you can see, I have imported a few functions import random matplotlib networkx and NumPy.

NumPy is used to compute average; it is a nice package for scientific computations. I am not using it much except for the 24th line as you can see, I am computing the average here in 24th line ok. Let me go slowly. This function walk (n, p); all it does is it generates a random graph with n nodes and p vertices starts from a random vertex in the given graph G (n, p). By that I mean an addition graph with n nodes and p probability p. And then I start with a with a vertex v which is uniformly picked from the vertex z.

And then I keep walking on this graph and then count how much time it takes until I visit all elements of the vertex set ok. Take a look at this code. It is quite self explanatory and then I what I do is I call this walk function quite often with i ranging from 20 to 300 with probability 0.3. Take a look at the walk function, you will understand what I am doing here. And every time I get an answer, what is the answer? The answer here is walk of i comma 0.3, simply returns the number of attempts number of random walks you must

take until you finish going through all the given vertices of the graph with i vertices and p equals 0.3.

And then I append this to a list called z and repeat this experiment some 10 times and then take the average of the number of random walks it takes for me to reach the entire graph. And then I do it for graphs of the size 20, 21, 22, 23 up to 300. And, then I plot on the x axis this range namely the number of vertices in the graph and in the y axis this plots the list l which stands for the number of random walks you have taken to cover the entire graph. So, let me execute this program and see what happens.

(Refer Slide Time: 08:37)

```
import random
import matplotlib.pyplot as plt
import networkx as nx
import numpy

def walk(n,p):
    start=random.randint(0,n-1)
    G=nx.erdos_renyi_graph(n,p)
    S=set([])
    v=start
    count=0
    while(len(S)<n):
        Nbr=nx.neighbors(G,v)
        v=random.choice(Nbr)
        S.add(v)
        count=count+1
    return count

l=[]
for i in range(20,300):
    z=[]
    for j in range(10):
        z.append(walk(i,0.3))
    l.append(numpy.average(z))
    print i,"--->",numpy.average(z)
plt.plot(l)
plt.show()

~
~
:!python %
```

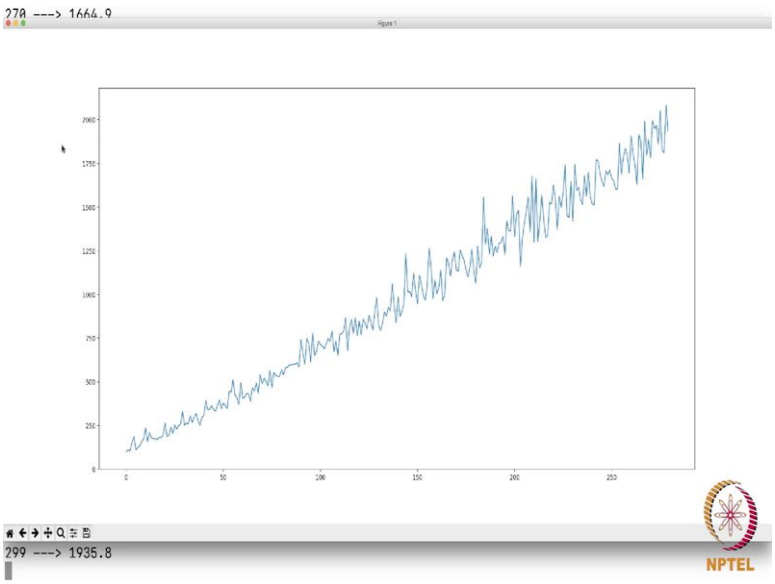
(Refer Slide Time: 08:39)

```
247 ---> 1449.6
248 ---> 1440.6
249 ---> 1647.8
250 ---> 1418.0
251 ---> 1743.5
252 ---> 1594.5
253 ---> 1612.7
254 ---> 1543.9
255 ---> 1513.7
256 ---> 1679.3
257 ---> 1560.9
258 ---> 1699.8
259 ---> 1556.9
260 ---> 1517.5
261 ---> 1512.9
262 ---> 1772.9
263 ---> 1763.5
264 ---> 1691.0
265 ---> 1646.6
266 ---> 1618.5
267 ---> 1706.6
268 ---> 1684.8
269 ---> 1712.6
270 ---> 1664.9
271 ---> 1652.6
272 ---> 1603.6
273 ---> 1684.9
274 ---> 1864.9
275 ---> 1687.6
276 ---> 1786.5
```



As you can see, the these are the number of nodes and the number of at random walks it takes for you to cover the entire graph. So, let us wait and see what happens. So, 263, 4, 5, 6, 7 so, on; it goes on till 300. So, as you can see the right side is the answer which is the number of random walks it takes for you to go to the entire graph.

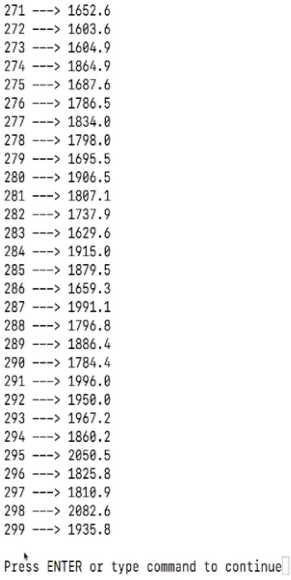
(Refer Slide Time: 09:13)



So, now I get the plot the plot looks beautiful as you can see um. The x axis is the number of nodes in the graph and y axis is the number of attempts it takes for you to number of random walks, you need to take to cover the entire graph.

What does the signify? It signifies that it is just a little bit of an effort a few times the number of nodes in a network is what it takes for you to cover the entire graph and that is a very nice observation that we make. As you can see the plot clearly says it is looking very linear, although it is not very linear a little bit of observation will tell you that it is very close to a n log n function. Anyway details a aside we will not worry much. All that I need to understand is this is very much possible.

(Refer Slide Time: 10:07)







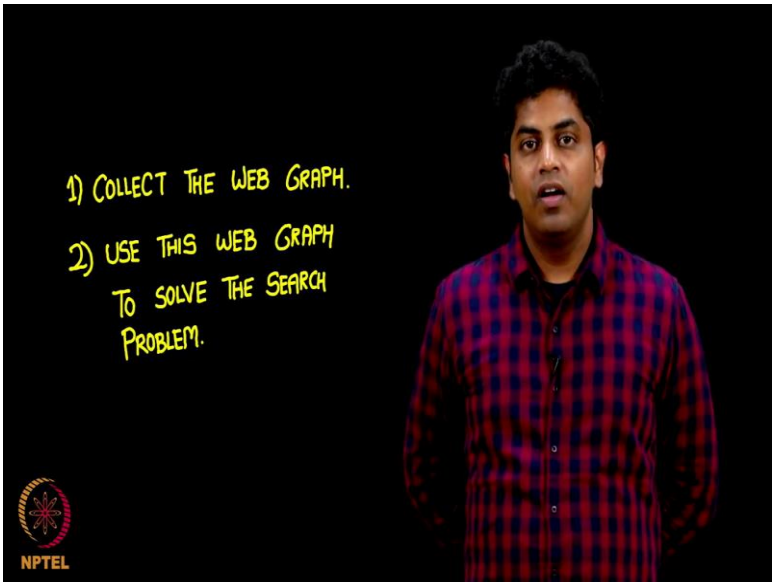
For example, when I take a node on 299 when I take a graph on 299 nodes, when I take a graph on 299 nodes in roughly 2000 random walks, I cover the entire graph. As you can see 298 seems to have taken a little more time as you know this is averaged. So, if you have average a lot more, this will be a properly increasing function.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Link Analysis
Lecture - 77
Equal Coin Distribution

Going with the flow, we had two questions.

(Refer Slide Time: 00:11)



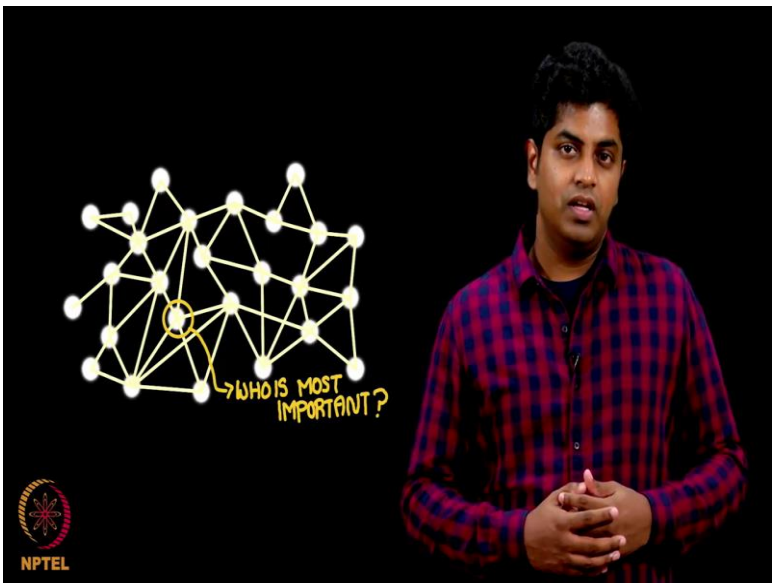
Question 1 was collect the web graph. Question 2 is use this web graph to solve your search problem. Question 1 you all are convinced is very doable, why? You saw the plot just now. When there are n number of nodes, it is not ~~very difficult~~difficult to explore the entire graph with n nodes. Random walk makes you search for all possible nodes and visit all the places that way you would know what is linking to what; not only will you get all the nodes; you will get a whole lot of edges as well right.

(Refer Slide Time: 00:45)



So, which means a partial structure of the graph is reveal to you. The second question that we asked how can we use such a web graph to solve our search problem? That we should go slightly slowly. I am going to divide my explanations in several pieces. I will go piece by piece; at the end put all these pieces together and you will see that we can solve the search problem just by using this web graph. So, I am going to go in bits and pieces and solve the problem of how we can use this web graph to solve the search question. The first piece is as and always a puzzle.

(Refer Slide Time: 01:43)



You are given 30 people with complicated friendship between them, you have to decide who is the most important person here and I am going to use a very weird strategy to find out who is the most important person. Repeat 30 node graph, they are all friends. I am going to find out a nice way in which, I am going to use a nice way in which I can find out who is the most important person here. What do I do? As I told you it is a very weird strategy; do not break your head on why ~~am I~~I am doing it.

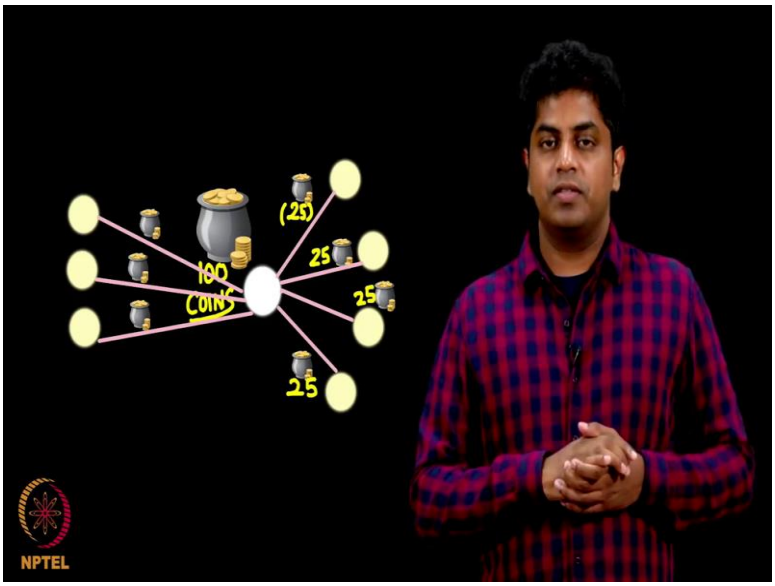
What has this to do with a question? As I told you I am going to go in bits and pieces. The first piece is the stand alone question the standalone question; I mean do not try to ~~related~~relate with anything else just look at this is a problem in its own right. 30 people and I am going to use the following strategy to find who is good here, who is the best here.

(Refer Slide Time: 02:37)



Basically, I am going to rank everyone who is the best, who is the second best, third best. What I do is I take 100 gold coins and give it to every single person on this network of 30 people. So, the first person gets 100 gold coins second person gets 100 gold coins 100 100 100 100 100 30 people; each of them get 100 gold coins. And what ~~is~~does the first person do?

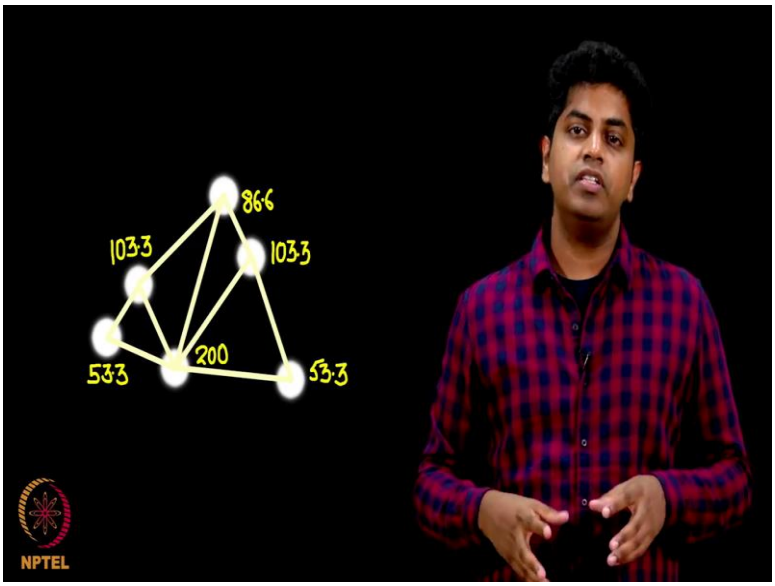
(Refer Slide Time: 03:15)



Assume a person has 3 edges. This node has 3; let us just take one node it has 3 outgoing edges. So, 100 can be divided into 33 33 33; I am going to round of ok. So, let us say 90 let us say 4 edges are going not 3, 4 edges are going 100 gold coins to a node and this node gives 25 gold coins to each of its neighbours. If it was 3, I will just divide it by 3 and 33.33 each. I will assume gold coins ~~can you can~~you can consider decimal part of it and then give it and by powdering it let us say.

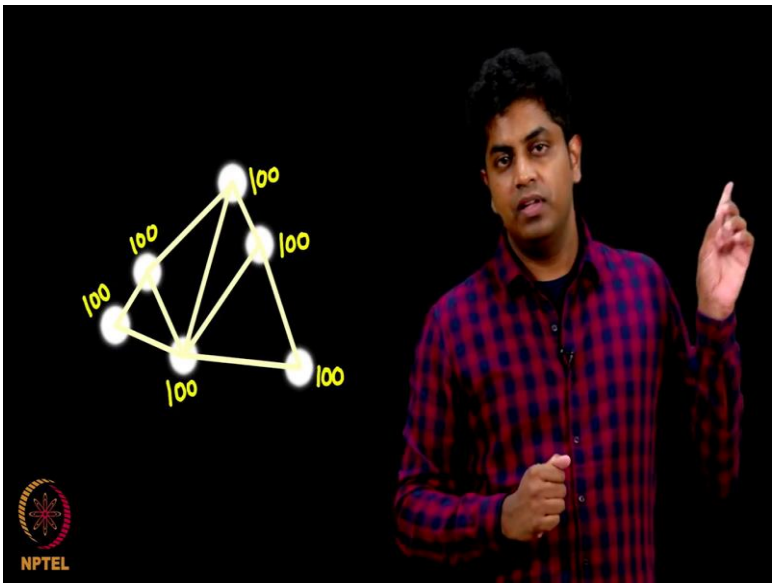
So, every person distributes the gold coins that they have to their neighbours. Now this one who has 4 neighbours to him will have some people who have him as a neighbour. So, he will also get some gold coins right.

(Refer Slide Time: 04:19)



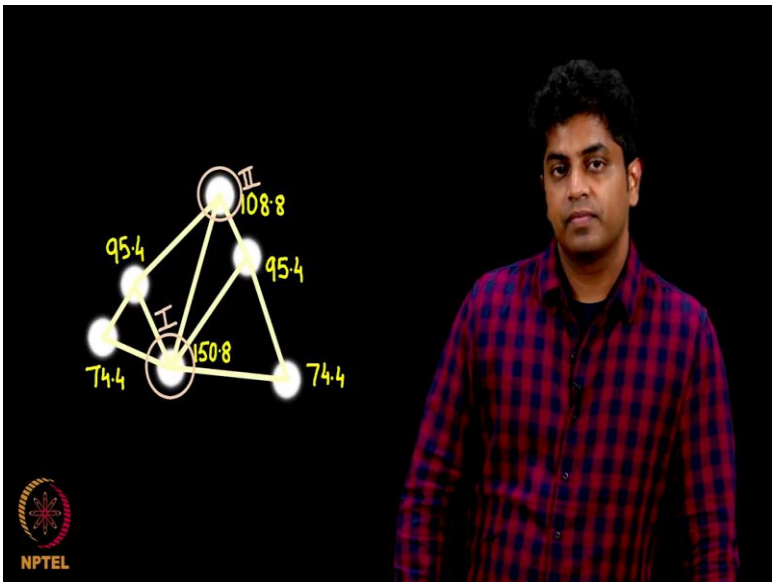
So, all I do is I take a network. As you can see 100 gold coins, each one iteration one snap like this will make everyone distribute their existing gold coins to all their neighbours.

(Refer Slide Time: 04:33)



You see with one snap, everything ~~change~~changes in the network on these 30 nodes. Another snap again it gets distributed. Observe very closely.

(Refer Slide Time: 04:37)



What is the distribution? Whatever gold coins you have, whoever are your neighbours you should just go and give it to them and whoever is pointing to you they will give their gold coins to you. In every snap this happens, every snap this happens. Keep doing this, snaps are basically iterations you see the graph changing; first snap, change; second snap, change; third snap, change. You see it keeps changing like this, keeps changing like this and so on and so forth. I keep doing this up to eternity guess what happens.

Very interestingly, this converges which means there is a stage after which the gold coins get accumulated and it does not change; just stays stagnant. And in that ~~state~~state, you go and see who has the maximum gold coins, you will see there is this node which has the maximum gold coins. You call him rank 1. Now what is it denote how did you accumulate so many gold coins? You did some random process, you simply started giving gold coins that you have what ~~is~~does this even denote in the first place?

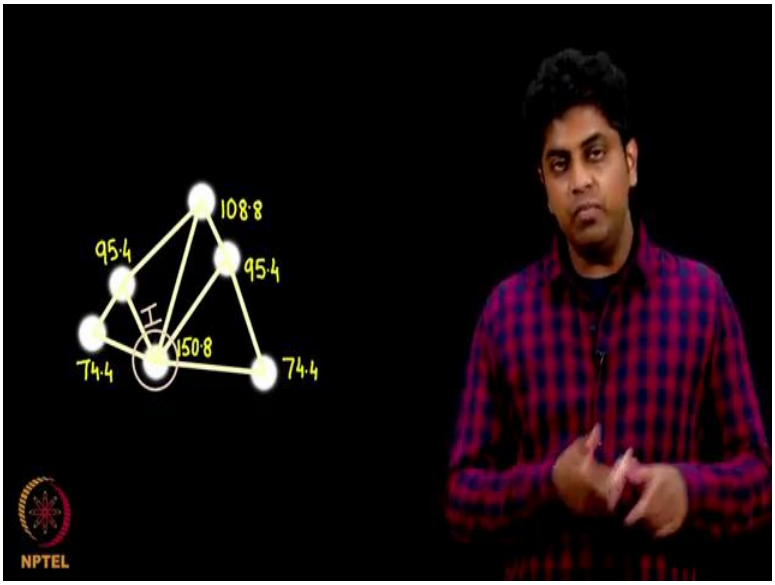
Let us see that in a while, but I just did this for fun, a very weird strategy to find out who is the most important person. Whoever accumulates the maximum gold coins eventually with time is the most important person. The second most gold coin collector is the second most important person; third, third most important person; fourth, fourth; fifth, fifth so on. What just happened? Pause and think about it.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar
Link Analysis

Lecture - 78
Random Coin Dropping

So, just now we saw a strategy of every node being given 100 gold coins and these nodes distribute all the gold coins to their immediate neighbours; equally. And as this process continuous and continuous the number of gold coins they get distributed not evenly although, we started evenly; they get distributed haphazardly.

(Refer Slide Time: 00:26)



And I go and see that node which has accumulated the highest number of gold coins, call him the winner first king; first top one person; rank 1 person. Second most gold coin accumulator, I call him the second best, second king I rank him accordingly. That was one way.

I will now give you yet another way of ranking the nodes. Now, I will not use gold coins; I will use something else of course, I will use gold coins, but I will give the gold coins in a quite different way.

(Refer Slide Time: 01:10)



What I will do is, I will stand on a node, some node and from 1 node assume I have 4 neighbours, I jump to a neighbour uniformly at random and give that neighbour a gold coin. From that node I will jump to yet another node, whatever it is pointing to maybe, it is pointing to 5 nodes. I will pick 1 node uniformly at random jump there, give him a gold coin. Remember the car and roads experiment I keep visiting the places I said, and I explore entire city, I am doing something similar here. From 1 node is just take a random hop and go to another node. Whenever I visit a node, I drop a gold coin on that node, I keep doing this, keep doing this, keep doing this, and I see, how many gold coins are accumulated by these nodes?

After some 1 million such random walks; I am walking randomly right. So, I call it a random walk. Random walk, gold coin drop; random walk, gold coin drop; random walk and I drop a gold coin. So, on and so, on and so, on and million times and I observe after a million iterations, every node has accumulated gold coins.

Although, they are not equal different nodes have accumulated different number of gold coins right. We can see that the figure says, if I take 1 million random walks which node gets how many gold coins? At the end I again, look at which node got the highest gold coin? I observe that the node which got the highest number of gold coins here was the same node which got the highest gold coins in my previous strategy previous experiment.

In fact, the second best here will also be a second best there. In fact, third best, fourth best, fifth best, all of them here we will agree with everything else there. It is very surprising that this is true for you, but mathematically if one sees it is not at all a surprise. In fact, as part of the advanced material for this chapter I am going to cover the mathematical details of why and how this is happening as well.

You are free to skip that its sort of it is not required that you go through it, but if you are curious to know what is happening, here where these 2 things the same, you may want to take a look at the mathematical material that I am going to record in the next few days, I mean in the next few hours of the video well.

(Refer Slide Time: 03:59)

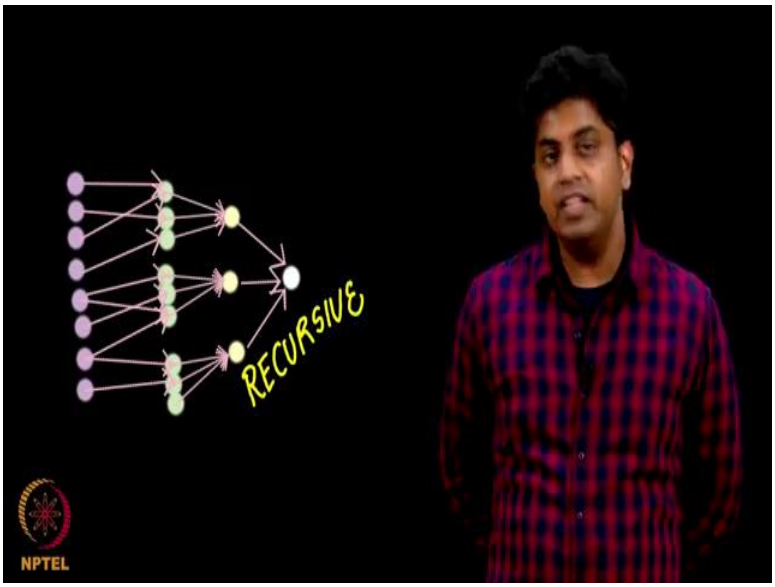


All that you need to buy from this explanation is that the strategy of 100 gold coins being distributed equally versus taking a random walk and dropping a gold coin, both this strategies rank the nodes in a same way; in exactly the same way. So, the moral of the story is sharing gold coins game and dropping gold coins game yield the same ranking of the nodes. So what? I am going to analyse this first game and the second game properly, for reasons of better explanation I am going to call the first game. The equal sharing of gold coins game, the second game the random walk drop the coin game. So, simply speaking equal sharing and random walk; game 1, game 2.

Let us slowly analyse and understand what these 2 games mean. What has happened so far? 2 games; this game is same as this game in terms of ranking the nodes. I will first

look at the first game which is 100 coins and equal sharing. Let us just think what happened here? What kind of nodes will attract the highest number of coins? A node has highest number of coins, if it is getting a lot of coins from its neighbours.

(Refer Slide Time: 05:19)



When will the neighbours give this a lot of coins? If the neighbours are getting a lot of coins. When will the neighbours get a lot of coins? If they are adjacent to whatever if there pointed by neighbours who get a lot of coins. You see this is a very recursive argument here. This captures the very notion of the following definition: you are famous, if famous people say you are famous, and those people are famous if famous people say that they are famous so on and so forth. This goes on like this correct.

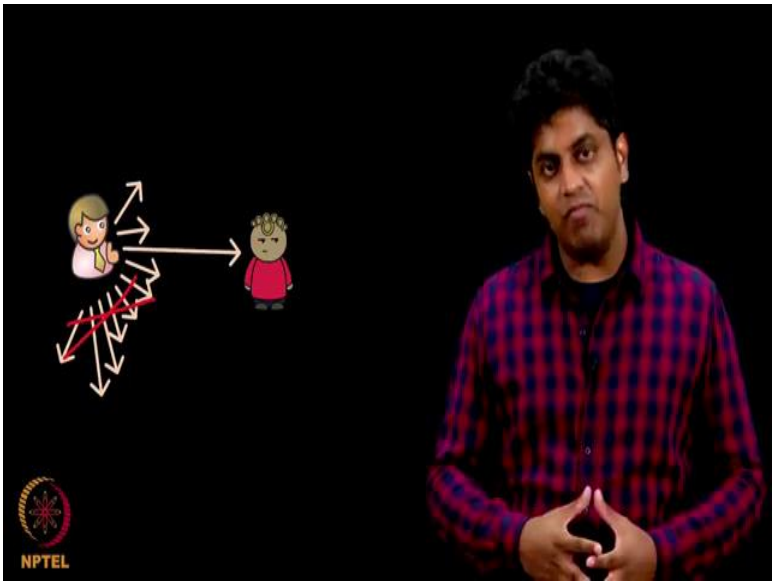
We discuss this in the introductory lecture in the first chapter right. You are famous if someone famous says you are famous. It is a very recursive definition. So, you see there is a definition for there is a abbreviation called GNU. If you are to computer science, you will know what is GNU. GNU stands for GNU is not UNIX and now what does that GNU stand for? Again, GNU is not UNIX; this is called a recursive definition.

For fun they have called it this way, but you see the all the all the time trying to say is the definition is sort of recursive here. You are famous if someone famous says you are famous and how do you know they are famous? What do you, what do you mean by famous then? Famous they are famous if someone famous says they are famous. And this famous is defined very recursively. Now you see what is happening in the first game. A

node accumulates a lot of gold coins if a lot of people, the people who are adjacent to it accumulate a lot of gold coins right. Only then you will the pass on the gold coins to me.

Please note, for me to accumulate a lot of gold coins people who point to me should accumulate a lot of gold coins and they should not have a lot of people adjacent to them. Now, what do I mean by this?

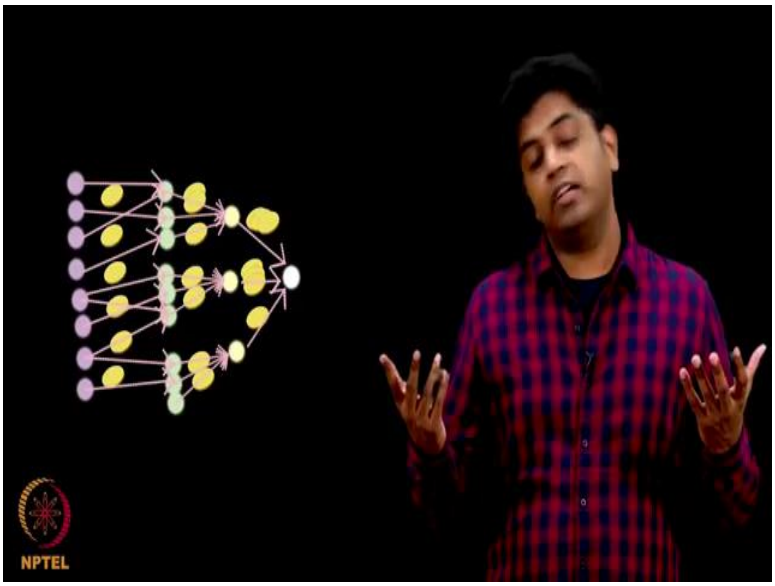
(Refer Slide Time: 07:28)



I have a rich cousin brother who helps me out whenever I want money let us say and I have 1 such cousin brother, but assume he has 10 such people to whom he is ready to help. Then I may not get all his help because his help sort of gets divided into 10. So, there are 2 aspects here; you should be having some good backups like a rich cousin brother to help you out in times of distress and he must not be ready to help a lot of people.

So, he must have very few friends outgoing, and he must be rich. When will he be rich? When a lot of people point to him, so, a rich person, a rich node is being pointed at by a lot of people, lot of rich people let us say. But that person should point to a few people and one of those few people should be you I am just set up dissecting this problem and then observing it. But what I am trying to say is the moral of the story so far.

(Refer Slide Time: 08:38)



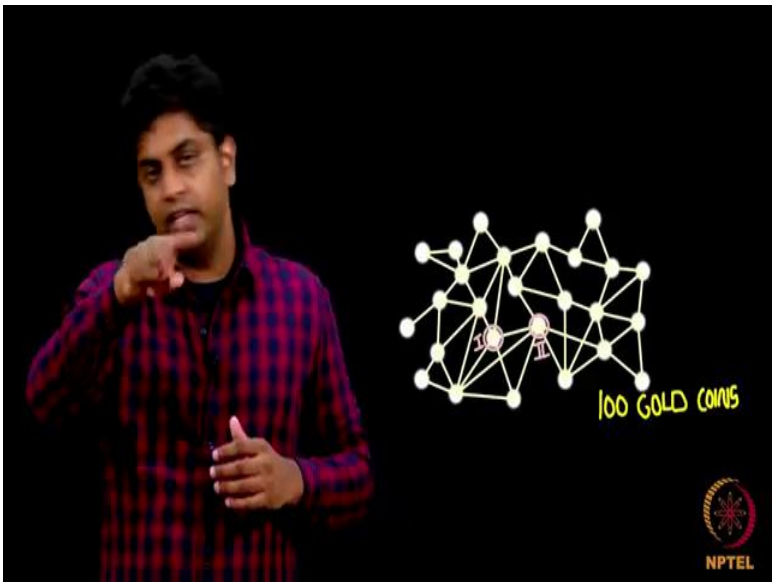
If you have understood that a node accumulates a lot of gold coins, if it is being pointed by nodes that accumulate a lot of gold coins. This much is enough and then you call a node to be highly ranked if it is being pointed by highly ranked nodes.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar
Link Analysis

Lecture – 79
Google Page Ranking Using Web Graph

Let us now, get back to the web graph question, 2 questions that we observed. We asked rather, how do you collect the web graph? How are you going to use it to solve your search problem? I am going to collect the web graph by taking random walks I collected. As you are collecting, as you are visiting nodes through random walks, you give points to each node.

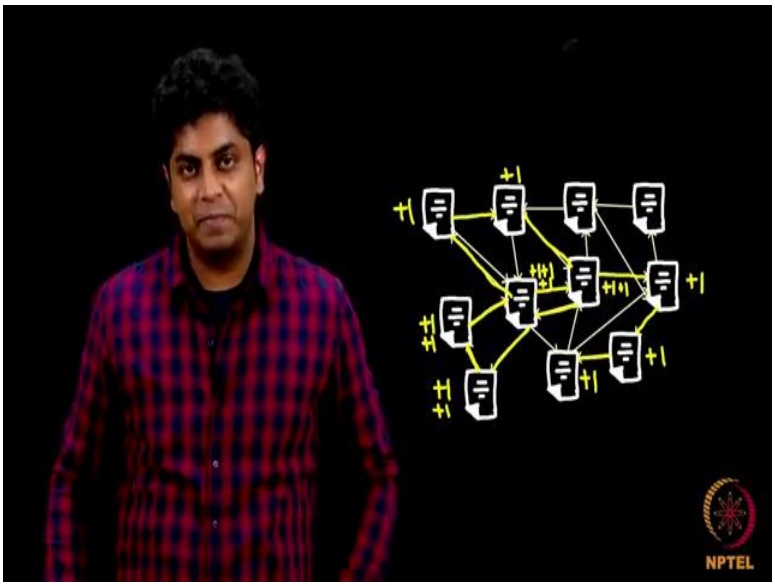
(Refer Slide Time: 00:36)



For example, on this 30-node network which we did in let us say, second game the random coin dropping game. We dropped the coins and we saw which node got the highest gold coins.

On the web graph we are going to do the same thing. We are going to walk on this web graph.

(Refer Slide Time: 00:53)



We keep walking, keep walking, keep walking, we walk like let us say billion times and keep dropping the coins and we look at which node has the highest coins. And when you search for a keyword in the search engine what Google does, is this. It firstly, keeps walking on the web that is called crawling; it keeps crawling on the web keeps dropping the gold coins and sees which node has the highest gold coin.

Every single page has basically, the gold coin accumulation points ok. When you type in let us say a keyword namely ah, I will say Britney Spears; if you type in the keyword Britney Spears we all know she is very popular, very popular pop singer. There is a there are lot of pages which talk about Britney Spears, which page should come first? What Google will do is it will go and retrieve all those pages, which has anything to do with Britney Spears. You will have some 10000 pages maybe, even more and as an example let me say 10000 pages.

Now, Google does not display in fact, it displays all the 10000 pages, but it should display the most relevant page in the beginning. What is the most relevant page? That page, that has accumulated the maximum gold coins; this is how Google works. Google just displays all these 10000 results in descending order of the points that page is accumulate as Google crawls the web and drops a gold coin, on each page that it visits. Very simple, very straight forward, very easy to understand algorithm, very easy to implement technique it works like a charm.

Google harnessed the potential of what is called a web graph, where web pages are related by hyperlinks and Google as an industry thought one should as a before it became Google, in fact in fact, story is very interesting, these 2 people Larry Page and Sergey Brim, they were sitting in a old garage and they were trying to implement this; this particular algorithm of crawling through the web. In fact, their crawling was also very suspicious that people thought that they are trying to hack something and things like that, in fact, they gone into trouble because of this. You can read up the literature for this.

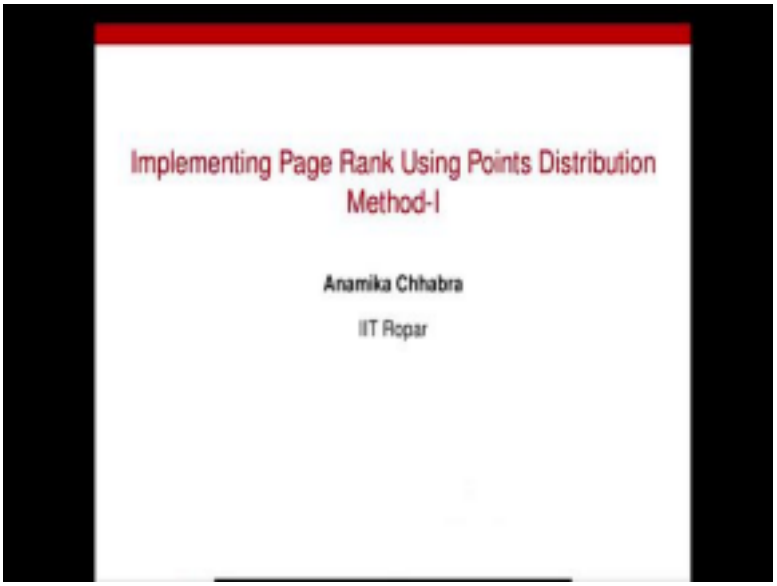
I mean articles on this, in fact, there is a nice documentary made on this as well. I strongly suggest that you watch it. So, these people were doing something out of the box, which was like crawling the entire web and then giving points to each node and using that to give you the right search results. Work like a charm something that is unexpected which is the web graph seemingly useless of what use you think a person like me and you, remember when we had this coffee table discussion in 1990, do not you think? By no means one could think of seemingly useless ah data like the web graph coming in use in a problem such as search.

And that seemingly useless data made this industry a big billion-dollar industry for a simple reason that it avoided hiring thousands of people which is like the first thought that one will get when it comes to solving the search problem. All that they did was take the web graph. How easy is to take? It is easy to take just crawl, take random walk; you will get all the web graph. Not only will you get as you crawl, as you take random walk, drop a gold coin and see which page has how many gold coins. And then when someone asks for a keyword search and show all the pages which has this keyword but sort them according to the gold coins accumulated by this web pages and that is precisely what Google does.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar
Link Analysis

Lecture – 80
Implementing Page Rank Using Points Distribution Method – 1

(Refer Slide Time: 00:06)



Hey everyone, in this video we are going to implement page rank using points distribution method. I assume, you have been already introduced to this method in the previous videos. So, I will just be briefing out the technique and then we will start the implementation. So, in this method what happens is, we start with the signing of fixed equal number of points to every node in the graph. After that every node distributes its points to its neighbours, by neighbours I mean the nodes that are connected to this node by an out link.

So, basically what we do is we take a node and we look at its out links and the nodes we are connected to this node through an out link get an equal share of points from this node. For example, if there is node which has 100 points and it has 2 out links; so, basically it has 2 neighbours. These 2 neighbours should be getting 50 points each from this node. That is, that is called points distribution in case the node is having 4 neighbours and its having 100 points in that case every neighbour will be getting 25

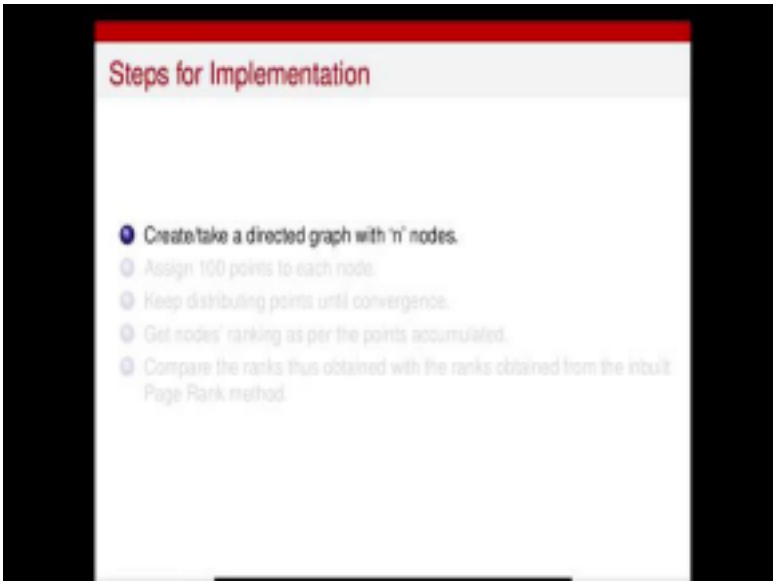
points each from this node.

Now, that point distribution happens for every node in the graph that is called one iteration. So, every iteration, so, after every iteration the points that every node contains keeps changing. For example, initially if we assign some 100 points to every node and we perform this points distribution after one iteration, the points that every node contains will change. And we keep repeating this process and after every iteration the points will keep changing; however, after some point after some number of iterations what happens? The points do not change, that is even after distributing points the points that the nodes were containing earlier is same as the points that the nodes will be containing after the iteration.

That happens because the number of points that the node distributes is equal to the number points and it gets from the nodes through which it relates to an in link. So, we basically get a convergence, and as soon as we get the convergence, we stop. And this point every node will be having some number of points after attaining the convergence, every node will be having some points and we can sort the nodes based on that that point distribution. Basically, we can rank the nodes and that ranking is called page rank.

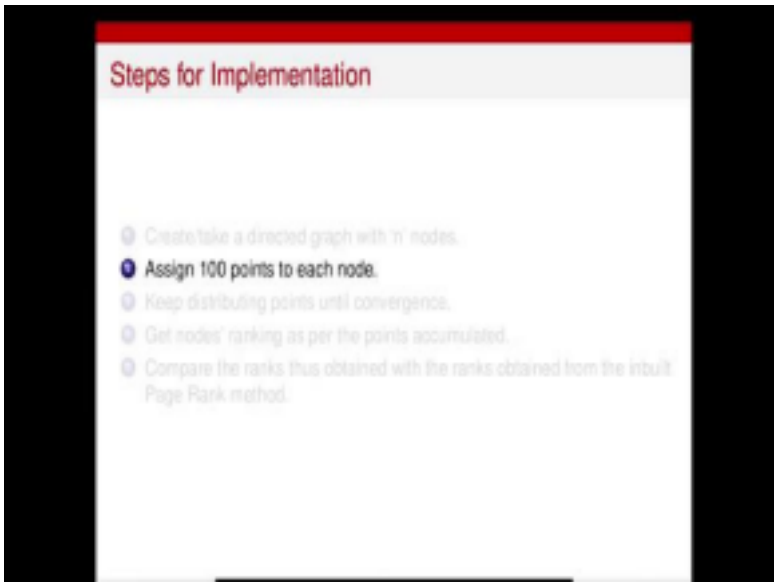
So, that was about the points distribution method. Now let us see the steps that we are going to follow for the implementation.

(Refer Slide Time: 01:49)



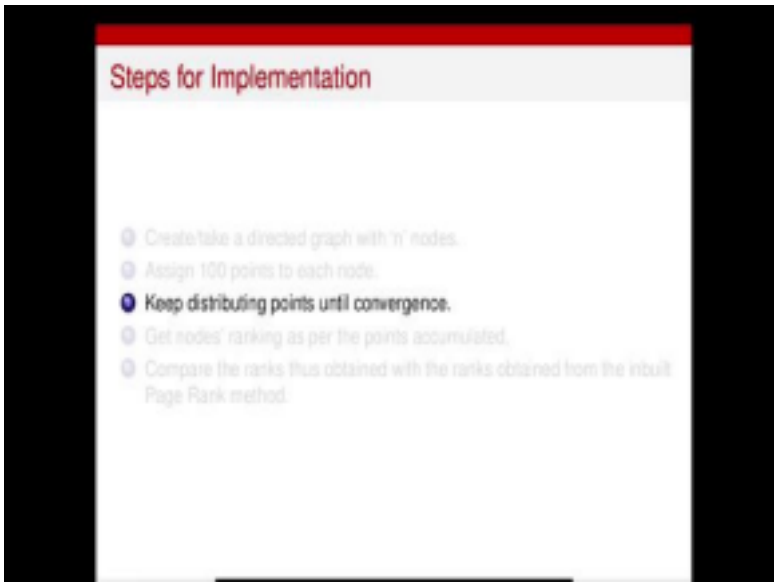
So, as a first step we are going to take a directed graph as you might be knowing for page rank it is better to take a directed graph because the points are distributed based on the out links. So, we are going to create a directed graph, we can also take make use of a function from networkx to generate the directed graph, but we will be creating a graph here by ourselves.

(Refer Slide Time: 03:19)



After that we will be assigning 100 points to each node. So, that is the initialisation of the points to every node. As I told you after that every node will distribute the points.

(Refer Slide Time: 03:30)



So, as a third step every node will keep distributing its points to its neighbours and this process will keep repeating until, we get a convergence that is, until a point where the nodes points stop changing, that is where we will stop.

(Refer Slide Time: 03:49)



Now, after that we will be getting some distribution of points amongst these nodes and we can rank the nodes based on the points that they have accumulated. That ranking will be called page ranking; page rank basically, to validate whether the method is working fine or not.

(Refer Slide Time: 04:08)



What we will do is as the last step is, we will make use of the inbuilt page rank method from network x and we will see the ranking that we get from that method and we will compare the ranking that we got from our implementation. So, these are the steps that we are going to implement. Let us get started with implementation.

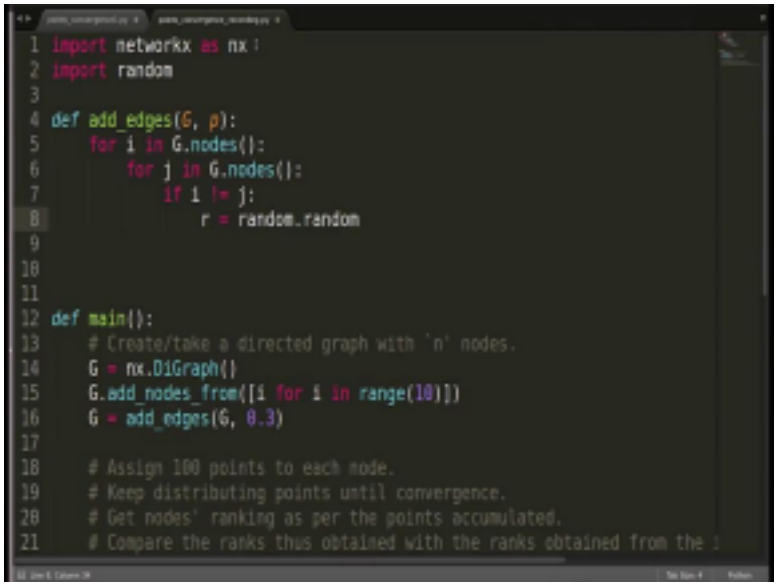
Social Networks
Prof. S. R. S. Iyengar

Department of Computer Science
Indian Institute of Technology, Ropar
Link Analysis

Lecture - 81
Implementing Page Rank Using Points Distribution Method – 2

Let us start with the Implementation of Page Rank Using Points Distribution Method, as we discussed in the previous video.

(Refer Slide Time: 00:12)



I have pasted to 5 steps from the previous video over here and we will take the steps one by one. So, let me create a main function here ok. So, let me import networkx first. So, let us take these steps one by one. So, the first step is create or take a directed graph with n nodes as I told you previously. So, we can use a generator for creating a directed graph that is some function from network, we can make use of. For example, we have a function gnr graph which gives us a directed graph. We can use any function, or we can create our own function to create a directed graph as well. In this video we are going to created at directed graph ourselves.

So, let us see how we can do that. So, as a first step I am just creating an empty directed graph by using this function G = nx.DiGraph ok. This graph is empty, but it is a directed

graph. Now, let us add the nodes and then we will add the at the edges. Let us add some nodes to it. So, I will write G.add_nodes from. So, we will add a list, I will write i for i in range, let me add some 10 nodes here. We can add more as well, but I want to show you the conversions, if you took take and a number of nodes it will take more nodes, it will take more time for convergence; you can always change that. So, I am taking 10 nodes at this point.

Now, we have added the nodes, we must now add the edges. So, I am going to use a technique of course, you can use many different methods to create a graph. The method that I am going to use here is that I will be randomly adding edges between the nodes, that is also well known technique to create a random graph which I think we will be discussing in the next weeks video. But I will be just briefing out the technique here because we will be creating this directed graph using that technique.

So, for that purpose I will be creating a function, let me create a function add edges ok. I will pass the graph here which has nodes and no edges as of now. So, I will be probabilistically adding the edges, I will briefly explain you the method as well. I am passing a probability value of 0.3 here and this function will be written in graph where the edges have been added.

So, let me create this function here I will create it here so, I will write. So, as a parameter this function requires the value of p which is a probability value. So, let me tell you what we are you doing here. So, we are going to take all possible edges that can ever we added to the graph and we will toss I mean, and we will add these edges to the graph after tossing a coin. By that what I mean is we will take an edge and we will toss a coin, if a coin turns out to be head we will add that edge and if the coin turns out to be tail we will not add that edge.

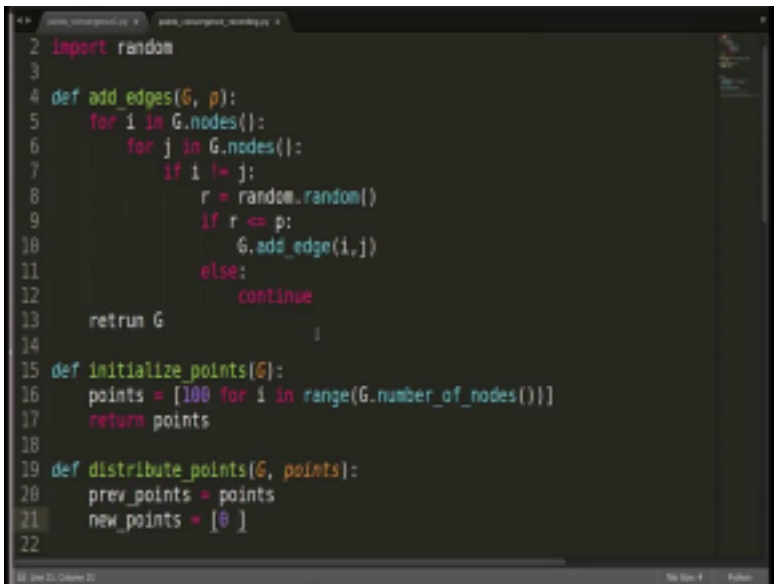
So, here in this case as you can see that we are passing a probability value p here and the value of p we have passed as 0.3; you can pass any value here, you can pass any value between 0 to 1 over here. So, by passing this probability value what we mean is that, you can assume that the coin is biased by a probability 0.3 which means with the probability 0.3 the coin will turn out to be head. And, with the probability 0.7 the coin will turn out to be a tail.

So, we will take an edge, we will toss the coin which is biased by this probability value and if a coin turns out to be head, we will add that edge otherwise we will not add that edge. So, this is the technique that we are going to follow. This is a well-known technique which is used to basically create random graphs. We are using this technique; you can use any other technique as well.

So, let us see how we can implement this. So, we have to take all possible edges, how can we do that. I can start loop I can write for i in G.nodes. So, I got all the nodes here because, we want the edges I will start another loop here. So, I will write for j in G.nodes ok. So, i comma j is we will give us all possible edges, we are now going to add self edges. So, I am going to check over here whether i is equal to j or not. So, I will write if i != j only then go ahead.

So, we have to implement that coin tossing here now, can you imagine how we can do that. We are going to take a random value for that we need to random package. So, let me import that, import random. So, we are going to make use of function random.random which gives us a random value between 0 and 1. Let me store that here r = random.random.

(Refer Slide Time: 05:56)



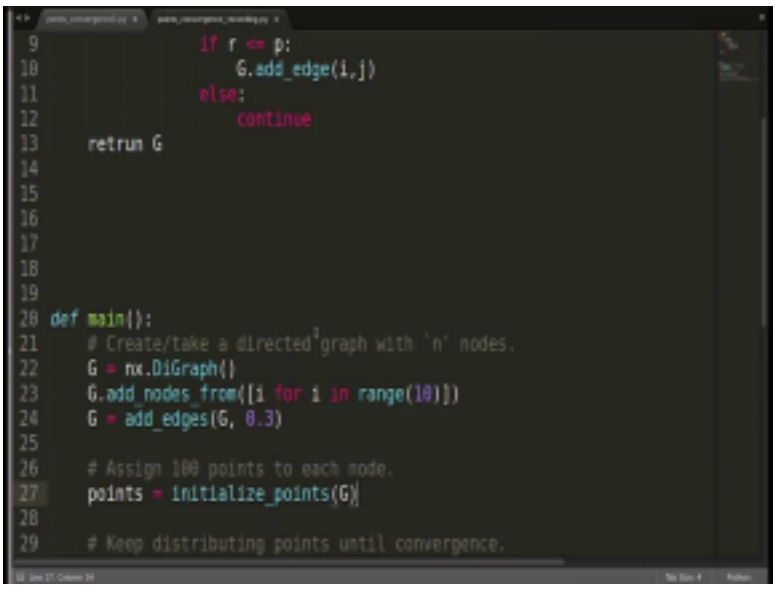
Now, we got some value here in r, we will check whether that value r <= p value or not. If that if r <= p; that means, we got a head and if r > p ; that means, we got a tail. So, that is that how we are going to implement that coin tossing part. So, we got this random

value, let us check whether r <= p; if it is we are going to add that edge. So, I will write G.add_edge, the edge is going to be (i , j) right and the edge part when r > p, we assume that that is that is a tail.

So, we will not add that edge, we will just continue here. So, this is how we are going to add the edges. Now, you know that this G is a directed graph. So, the edges that are going to be added here by using this add edge function are going to be directed edges. So, after this loop we would have added several edges, then we can return this graph. So, this is our add edge function and now we have created this graph G and we can go ahead.

So, we are done with the first point, let us implement the second one. It says assign 100 points to each node. So, we have to maintain a list points which will be storing the points for every node and we have to initialise this points list to 100. Let me create a function although you can directly write it, but I am going to create a function for this.

(Refer Slide Time: 07:35)



So, I will write points is equal to initialize points and I will pass the graph here ok. So, let me create this function ok. So, this function is just going to create a list points and that points list will be having 100 points for each node. So, I will write 100 for i in range, what is going to the size of this list. And, the size of this list is going to be equal to the number of nodes that are there in the graph although we know that there are 10 nodes as of now, but we can always change that. So, let me generalized it by writing G.number_of_nodes and then we should return this list points we are done ok. Let us go

here; we have done the second point. So, every node has 100 points right now.

Let us go to third step, third step says keep distributing points until convergence. Now, how do we distribute_points? Let us see how we do that. For that purpose, let me create a function distribute_points. So, I will write define distribute_points. So, this is going to be one iteration and we are going to distribute_points in this function. As you know that after the nodes distribute their points their points change. So, we are going to need previous points and new_points.

Let us pass the graph here because, we are going to need that and let us pass the points that are there as of now. As I told you previous points and new_points let me create to let me create two lists here. So, I will write previous points to be the same as points and this is just for a ease of understanding. Another list I will create new_points, this is what we will be returning. So, this list I am going to initialize to 0 and during the distribution part, this list will get more points and that is what we will written.

(Refer Slide Time: 09:58)

```
                continue
    retrun G

def initialize_points(G):
    points = [100 for i in range(G.number_of_nodes())]
    return points

def distribute_points(G, points):
    prev_points = points
    new_points = [0 for i in range(G.number_of_nodes())]

    for i in G.nodes():
        out = G.out_edges(i)
        if len(out) == 0:
            new_points[i] += prev_points[i]
        else:
            share = (float)(prev_points[i])/len(out)
            for each in out:
                new_points[each[1]] += share

    return G, new_points
```

So, for now I will write 0 for i in range G.number_of_nodes. I have not copied G.number_of_nodes ok. So, this is the initialization of the new_points list which will be returned by this function. Now, how do we distribute? As I told you for every node, we will look at its out links. So, let me start a loop at these points, I write for i in G.nodes. Now, i is a node and we must distribute its points. How do we distribute? We look at its out links right. So, let us keep a track of the out links of this node i, the function that we

can use from networkx for this purpose is out_edges.

So, I will call that function I will write G.out_edges. Now, this function is there for directed graphs only, the parameter will be the node. So, we write G.out_edges. What this function does is, for the node i it will return a list of the edges which are out_edges from this node i. Now after this, this node i has to distribute its points and equal share of its points to its neighbours right. So, we have the points that this node i has in this list previous points right. We are going to distribute these points equally amongst its neighbours.

And the number of neighbours we can get by getting the list and by getting the length of this list. Now, what happens what will happen if the list is empty, that is the node has no out link. In that case, the length of this list will be 0. So, it is a good idea to check for that condition over here. So, let me check if the length of this out is 0 not, because in that case division by 0 exception might come. So, it is better to check it here.

So, I will write if length of out is equal to 0 which means this node has no out link ok. If a node has no out link, what should happen? It will not be distributed; it will not be distributing any of its points right. So, what we write is, new_points for i are going to be plus equal to I will tell you why I have written plus equal to previous points i ok. So, I have written plus equal to because, this point is going to retain its points it is not going to give its points to anybody because there is no out link. However, it might have some in links and through those in links it might be getting more points.

So, that is why we have to keep adding it. So, we have to write plus equal to. Now, this is what we will do in case a node has no out link. In the general case that is the nodes have out links, what are we going to do? We are going to distribute an equal share of these nodes points to its neighbours. Now, what is that share going to be? Let us create a variable share. Now, the share is going to be the number of points that are going to be distributed to every neighbour. What is this share going to be? So, as I told you if there are if there are 3 neighbours, the points the points that are going to be distributed is total divide by 3.

And what is the number of neighbours here? The number of neighbours is equal to the

length of out here right. So, what are we going to do is we will write previous points that is what is to be distributed right, you remember that you have to only distribute the previous points. So, previous points are to be distributed in equal share so, we divide by length of out this. So, this is what is to be distributed, let me sorry let me do the float casting here so, that we get the exact values ok.

So, this is a share that has to be distributed to the neighbours. Now, where are the neighbours? The neighbours are in this list; this list is a list of edges. So, for every edge 0th value is the source that is our node i and first value that is the value at index 1 is going to be the neighbour ok. So, let us start a loop here in order to distribute the points. So, I will write for each in out which means for every neighbour, we are going to update the new_points of every neighbours. So, I will write new_points for each now each is an edge.

So, I will write each 1, each 1 is going to be the neighbour. So, new_points plus equal to share so, that is how we will be updating the points of every neighbour. I think we are done, let us return. What are we going to return? We are going to return the graph G and we are going to turn the list new_points ok. So, we have created this function, this distributes points which is going to be one iteration where every node will be distributing its points. Let us go back to main and let us see what we have to do.

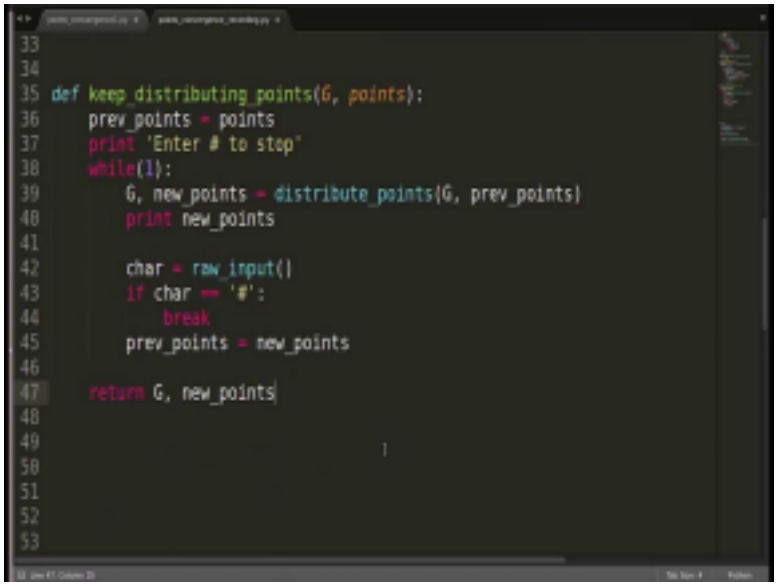
(Refer Slide Time: 15:22)

```
58
59 def main():
60     # Create/take a directed graph with 'n' nodes.
61     G = nx.DiGraph()
62     G.add_nodes_from([i for i in range(10)])
63     G = add_edges(G, 0.3)
64
65     # Assign 100 points to each node.
66     points = initialize_points(G)
67
68     # Keep distributing points until convergence.
69     G, points = keep_distributing_points(G, points)
70
71     # Get nodes' ranking as per the points accumulated.
72     # Compare the ranks thus obtained with the ranks obtained from the
73
74
75 main()
76
77
```

We have to keep distributing the points until the convergence. So, we have to basically

keep calling this function distribute_points. So, what we can do is let me create another function which will keep calling this function that we have just created. Let us call this function keep distributing points and the parameter will be G and the points that we have and this function should return G and points again. So, we are passing something and that is getting updated and that is what we will be returned. Now, what should this function do? Keep distributing points, this function has to keep calling the function distribute_points until we get the convergence ok. So, let us create this function ok.

(Refer Slide Time: 16:22)



So, as I told you this function has to keep calling this distribute_points function, let me start a while loop here. So, what I can write is while 1, they should keep going on ok. What should keep going on? Before that let me rename these previous points is equal to points, this is for the ease of variable name that is all ok. So, we are going to start this loop, this will keep going on and on. We have to call the function this distribute_points.

So, I will copy it and I will call it here, this function returns two things again. So, I will write G, new_points is equal to this right. What are we passing? We are passing previous points. So, let me rename these previous points done. So, we have called this function and one iteration is happened and we have got the new_points in this list new_points. If we want to keep track let us let us print this so, that we can see the updated points ok.

Now, this will keep going on and on, it has to stop somewhere basically when the convergence happens. So, you can do one thing, or you can do one or the two things, you

can either through your code check whether the convergence has happened or not that is one thing. And, other simple thing that we can do is we can keep observing as we are printing it right. So, we can keep observing these points and whenever we see that convergence has happened, we can stop it.

So, for that let us let us ask the user to stop it, enter may be # to stop and let us ask the user to enter a hash when he wants it to stop. So, we will take that through the function may be raw input and if that char is equal to # we are going to stop. So, I will write break ok. Now, after one iteration, we have to assign the new_points to be the previous points and we have to repeat the process. So, it is important to write previous points is equal to new_points and after that we will go back here, and the same thing will start alright.

I think we have done, let us return let us return the same two things G and new_points. I think this function should work ok, we can, may be check the code as well if it is working fine, wait there is an error here ok. So, let us check the functionality of this code whether it is working fine or not. So, let me call main here ok.

(Refer Slide Time: 19:34)

```
anamika@anamika-Inspiron-5423:~/NPTEL/WEEK_wise/WEEK_6_(Link Analysis)/Code/
Video_code$ python points_convergence_recording.py
Enter # to stop
[33.333333333333336, 58.333333333333336, 58.33333333333334, 33.3333333333333
36, 166.66666666666669, 116.66666666666667, 116.66666666666667, 158.33333333
333334, 75.0, 183.33333333333334]

[38.88888888888889, 70.1388888888889, 46.52777777777778, 31.944444444444446,
 195.83333333333334, 126.3888888888889, 120.83333333333334, 190.972222222222
23, 79.86111111111111, 98.61111111111111]

[42.12962962962963, 76.90972222222223, 55.84490740740741, 38.31018518518519,
 167.24537037037038, 84.95370370370371, 141.087962962963, 206.0763888888889,
 85.59027777777779, 101.85185185185185]

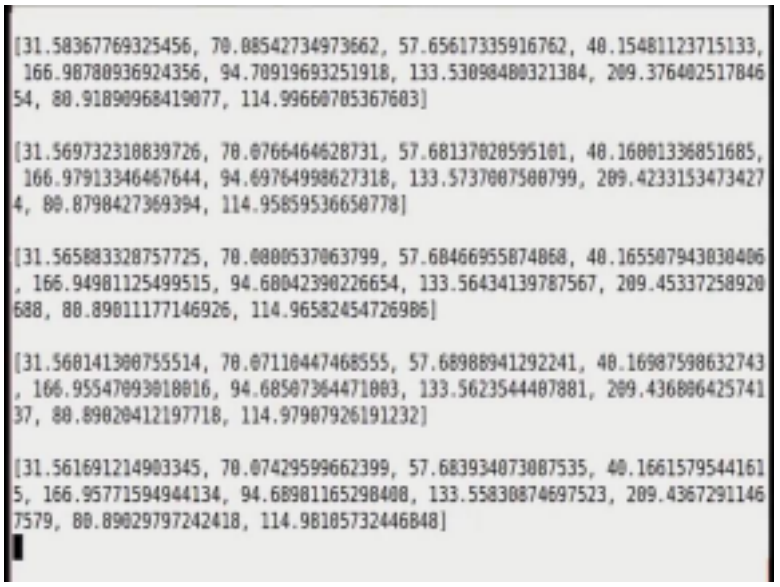
[28.317901234567902, 69.70968364197532, 60.595100308641975, 41.3676697530864
25, 161.23649691358025, 91.13618827160494, 137.7411265432099, 218.2339891975
3092, 76.62519290123457, 115.03665123456791]
```

Let us get back to our screen and let us try calling this file ok. So, these are the points, we did not print the initial points right. They were obviously, 100 100 each we can print that as well. So, this is these are the values that we are getting after one iteration.

And we have to press enter after it, after I press enter the next the next iteration starts.

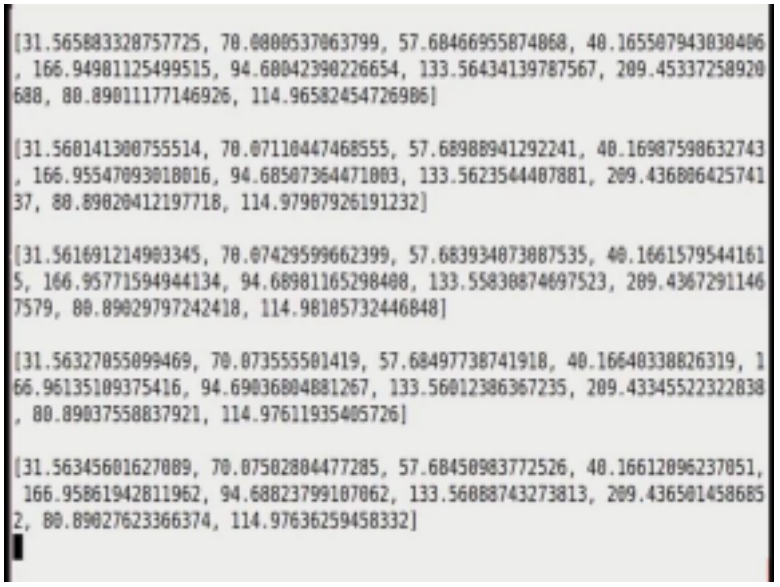
And, as you can see the points are changing the first node was earlier having 33 points and now it is having 38 points. I am keeping on pressing enter and we are seeing the points how they are changing.

(Refer Slide Time: 20:12)



So, I keep on pressing the enter ok, as you can see we are sword of reaching the convergence.

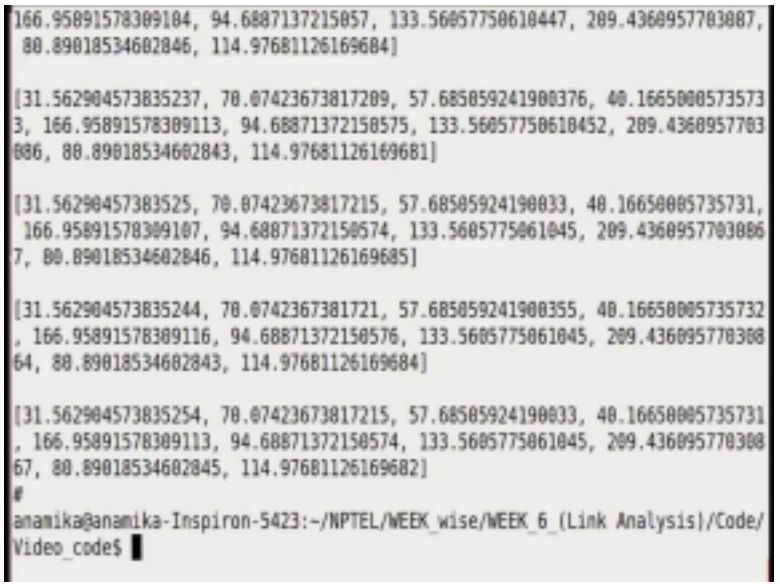
(Refer Slide Time: 20:21)



So, it may stop, it may happen that as you keep pressing enter it will not change at all. So, nothing will change, or it may happen that values are changing by very nominal

decimal points. So, you can stop wherever you want and or you can go on and on. So, it is up to you. So, I am keeping on pressing enter and I can see the values changing slightly only by decimal points. So, I can keep going on and on or I can stop.

(Refer Slide Time: 20:53)



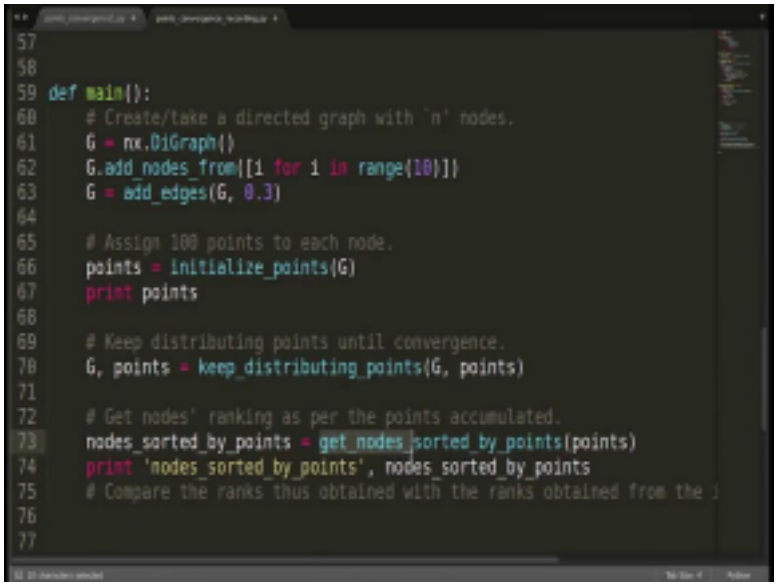
So, I want to stop let me press hash and we are stopping. So, these are the points distribution that we are getting, and we are sort of seeing the convergence taking place as well. Now, let us get back to our code and let us see what is pending ok, next step is to get the nodes ranking as per the points accumulated. So, we have to basically rank the nodes based on the points that they have acquired overtime after number of iterations, after the convergence point.

So, I think we will do that in the next video.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar
Link Analysis

Lecture - 82
Implementing Page Rank Using Points Distribution Method – 3

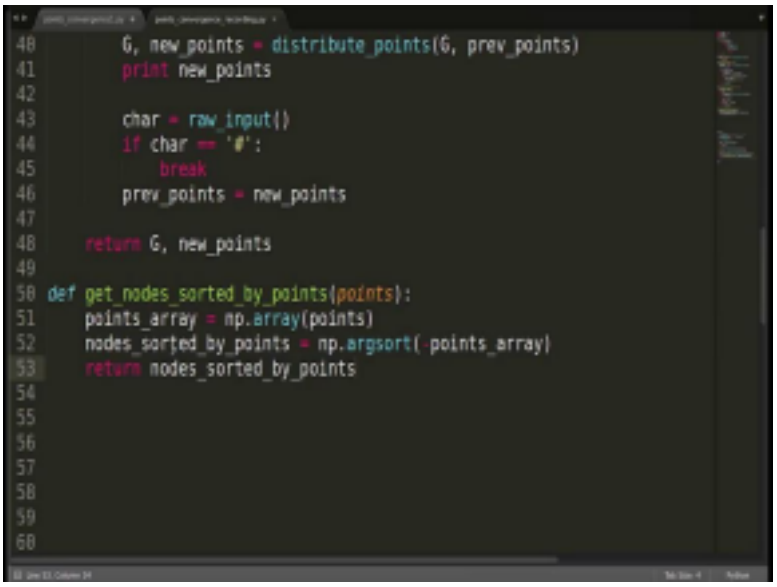
(Refer Slide Time: 00:07)



In the previous video, we saw the conversions taking place when it comes to the points that the nodes are able gather in every iteration. So, we saw that the points are converging. Now, the next step is how we can rank the nodes based on the points that they have gathered. Now, let us create a function to sort the nodes based on points. So, let me create get nodes sorted by points, I am going to pass the points list and it should return me the list of nodes. I am sorry I am sorry ok.

So, I will write nodes_sorted_by_points = get_nodes_sorted_by_points which is a function which I am going to create. And, then let me print the nodes that we will be getting. So, we will just be printing the nodes that we are getting, I should print the initial points as well. So, that we know how the points are changing. So, I am printing the initial points here ok, getting back to the function that you have to create get_nodes_sorted_by_points. Let us create this function here. Sorry ok.

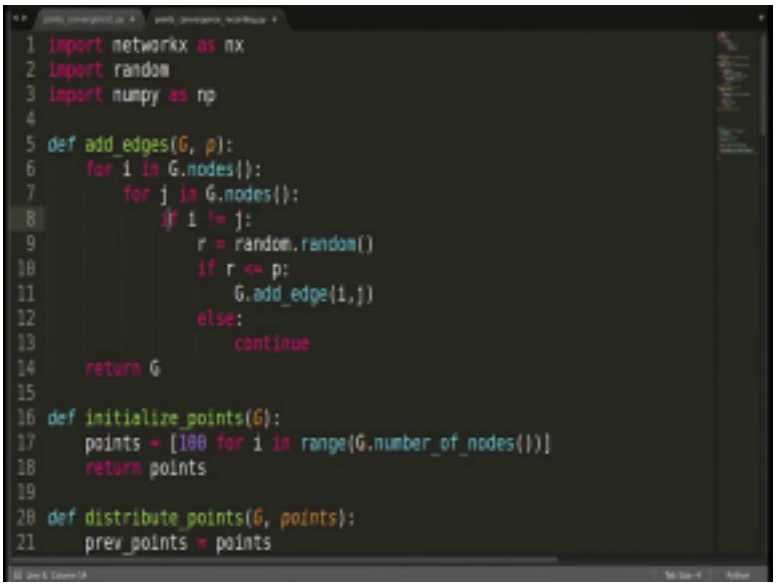
(Refer Slide Time: 01:27)



Now, what should this function do? This function has as an input a list which is points which is a list having the points for every node. And, what should this function do? This function should return a list of the nodes sorted by their points. Now, what are the nodes? How are the nodes labeled? The nodes are basically from as you can see here from 0 to 9 right; that is how the nodes are numbered.

So, we can just simply get the index of the nodes from this points list. So, basically what we have to do is, we have to sort the index of the values which are there in the points right. We can use again several methods for that, one method which might be very quick is we can make use of the package numpy. So, numpy has function argsort which basically returns the index, the indices sorted by the values of a list. So, we are going to use that function argsort that function is there in numpy.

(Refer Slide Time: 02:49)



```
import networkx as nx
import random
import numpy as np

def add_edges(G, p):
    for i in G.nodes():
        for j in G.nodes():
            if i != j:
                r = random.random()
                if r <= p:
                    G.add_edge(i,j)
                else:
                    continue
    return G

def initialize_points(G):
    points = [100 for i in range(G.number_of_nodes())]
    return points

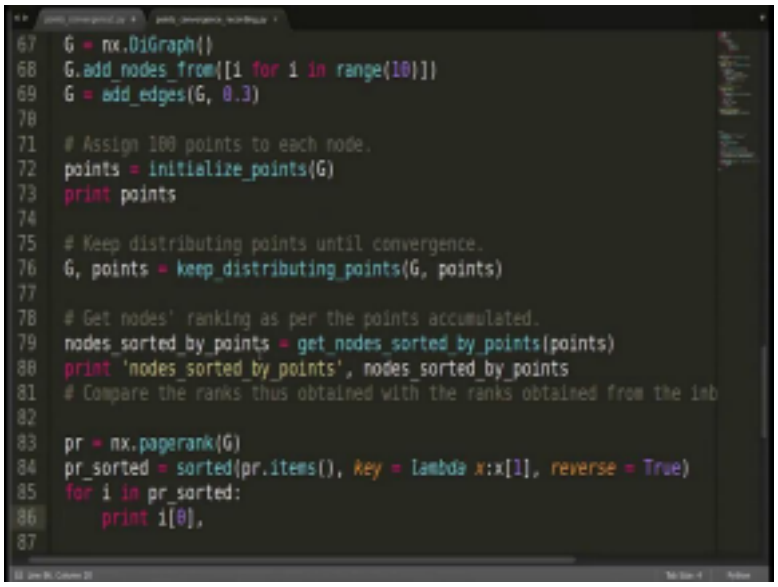
def distribute_points(G, points):
    prev_points = points
```

So, let me import that package. So, I will write import numpy as np ok. So, the input to wherever we yeah here so, the input to that function is basically an array which is a numpy array and here as an input we have a points which is a list. So, we have to convert that list into a numpy array. Now, how we do that? So, I will write points array is equal to np numpy.array. So, so this array is the function which converts a list into an numpy array ok. Now, points array is a numpy array and we can apply the function argsort on it. And, what is that function do?

It returns the indices sorted by the values of the list. So, write np numpy.argsort points array ok. What is it return? It returns the indices. So, indices basically are the nodes in our case. So, maybe I can write nodes_sorted_by_points equal to this thing ok. Now, one thing that has to be noted is by default argsort sorts in the ascending order right. And, which order do you want? We want descending order that is the node which has high points should be the first to be ranked ok; so, simple thing that we can do here just put a minus here.

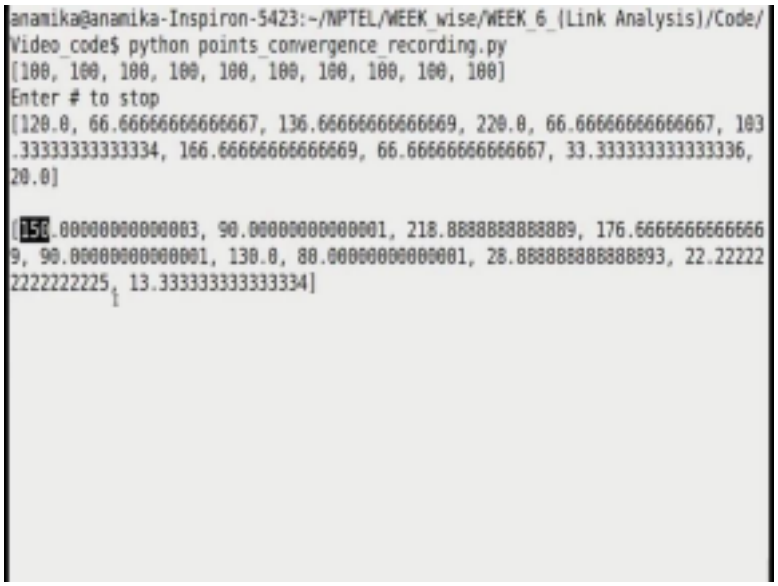
By putting a minus all the values are convert converted to negative and the ordering that we will be getting will be negative. So, as simple as that we can work with that simple method. So, this nodes_sorted_by_points is going to have the nodes sorted by of course, the points in descending order right. So, the first value will be the node which is having highest ranking and hence the highest page rank ok. Let us return this nodes_sorted_by_points, this should work.

(Refer Slide Time: 05:07)



So, let us go back to main and let us see is anything pending. So, we had already called this function and we had already printed the values. So, let us see whether this function works fine or not ok.

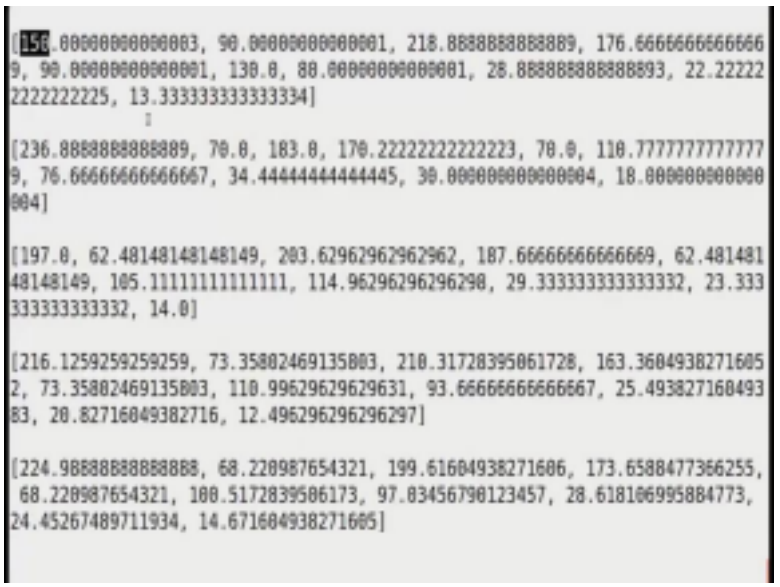
(Refer Slide Time: 05:18)



So, let us call this function, let us let us see how the code works. So, let me execute this ok. So, these are the initial points I am sorry these are the initial points 100 each, as we assigned initially. And then we started the iterations, after the first iteration as you can see the first node which was having 100 points is now having 120 points and second

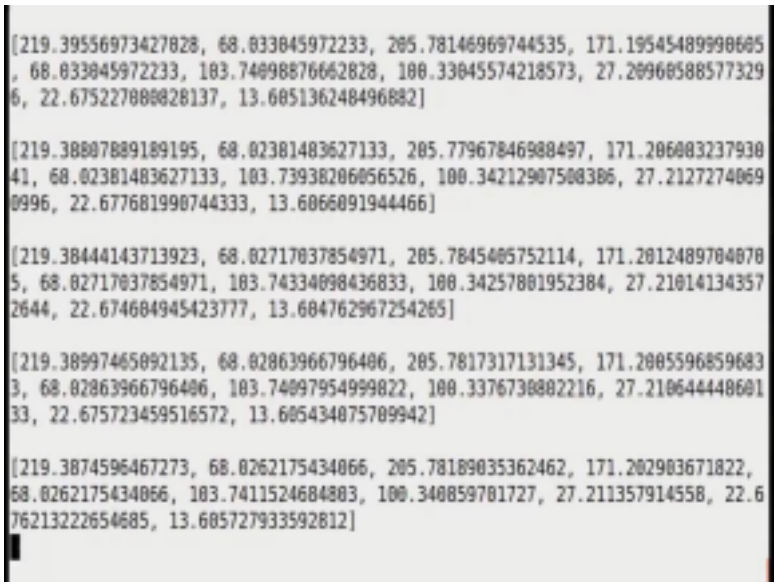
node the points for second node have reduced and so on. So, that that keeps changing; this is the scenario after first iteration. I press enter and this is this scenario after second iteration. So, the points have 150 for the first node, I keep on pressing enter.

(Refer Slide Time: 06:01)



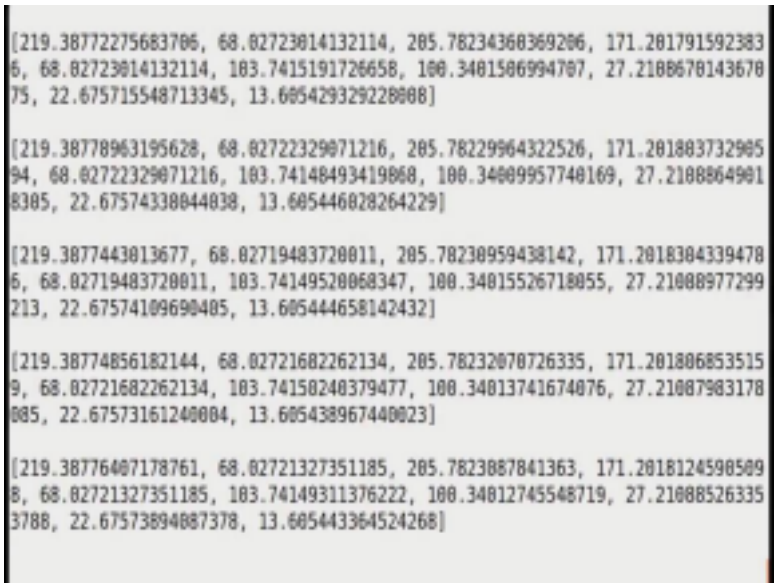
And I keep on seeing the values changing as you can see. So, let us concentrate on the first node as of now. So, 224 I keep pressing enter because it is keeping on changing it is 2 219.

(Refer Slide Time: 06:14)



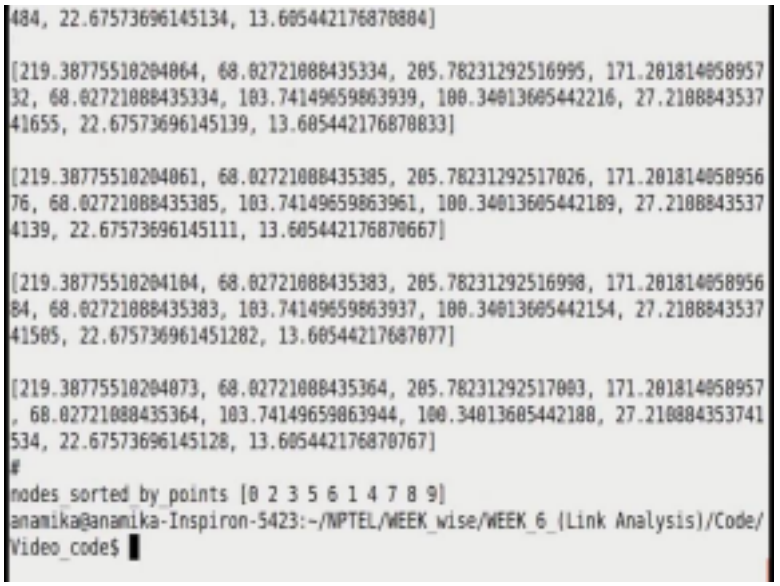
And only the decimal values are changing ok.

(Refer Slide Time: 06:20)



Let us let us look at any other node if the values are changing drastically. So, if the values are not at all changing, we will stop of course, if the values are changing slightly, we can either keep going on and on or we can stop there. So, that that should be enough and that is what I am mean to say; let us see if any other value is changing. I am giving on pressing enter and the values are only slightly changing. So, I think I can stop at any moment, let me press hash here.

(Refer Slide Time: 06:46)



So, our loops stops ok, this function is looking fine. So, I getting the nodes_sorted_by_points, these are the sorted nodes that we are getting. Basically, these are rankings; this is the ranking of the nodes. So, the node 0 is having the highest page

rank node 2 is on the second number and so on. So, this is the ranking that we obtained. Now, let us see whether this ranking is correct or not as per the page rank; by third that is there in networkx. So, let us get back get to the last step which is compare the ranks thus, obtained the ranks obtained with the inbuilt page rank method.

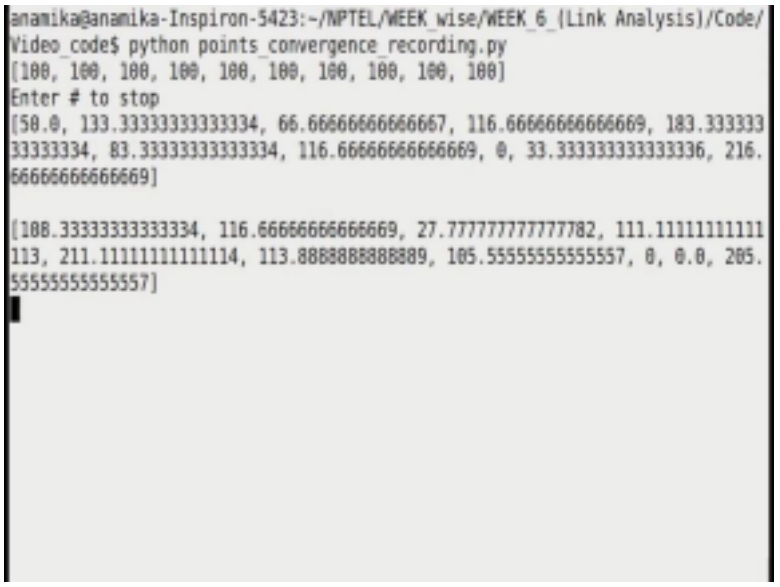
Now, let me tell you there is an inbuilt function which we can use to compute the page rank values for the nodes for a given graph. We have ourselves implemented it over here, there are several other methods as well let me tell you. We have implemented one of the methods which is using the distribution of the points. We might look at other methods as well maybe in the subsequent videos; let us check the ranking that we obtained from page rank. So, the function is nx.pagerank and we will pass the graph here ok.

Let me tell you what it returns, it basically returns a dictionary where the keys are the nodes and the values are the page ranks of the nodes. So, basically it does not tell the ranking of the nodes, it tells us the page ranks page rank values of each node. So, we would have to rank them ourselves. So, how can we do that? Let me sort this dictionary ok, we cannot sort the dictionary because the dictionary does not contain any sorting order. So, we can create a list of the tuples from the dictionary as we have been doing in the previous videos as well.

So, how can we do that? Let us see so, this is going to be a list of the sorted tuples based on the values. So, the function that we can use is sorted, I write pr.items and same old method that we use key is equal to lambda. How can how do we want to sort it? We want to sort it based on the value which is the second thing in the tuple which is x1. So, we will write x : x1 and by default it is a ascending right and we want it to be descending.

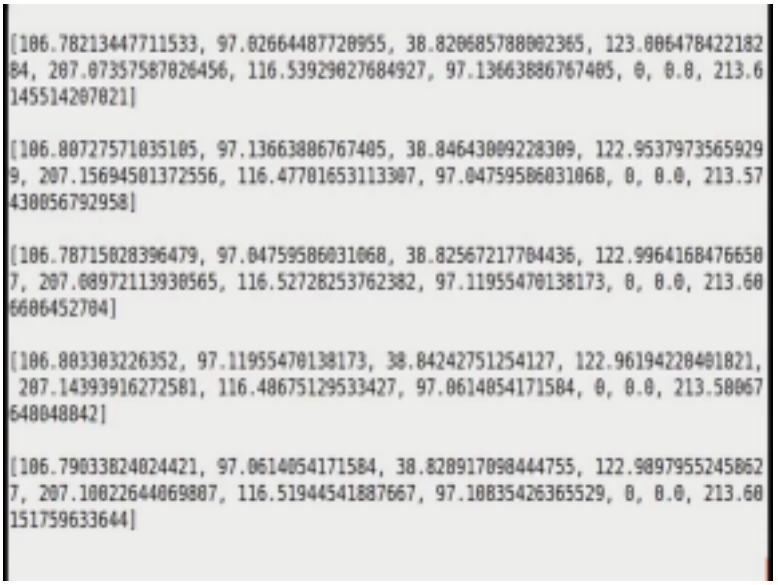
So, we will write reverse is equal to True. So, pr sorted is a list of the tuples ok. What we want? We want the nodes; we want the nodes sorted by the page rank value only. So, maybe what we can do is, we can only print the first value from each tuple. So, write for i in pr_sorted ok, I am going to only print the first value for every i ok. So, I want all the values in same line so, I have put a comma here ok. Let us check the working of the code. So, basically what we have to check is the ranking that we were getting r by r method and the ranking that we are getting by this method are the same or not.

(Refer Slide Time: 10:20)



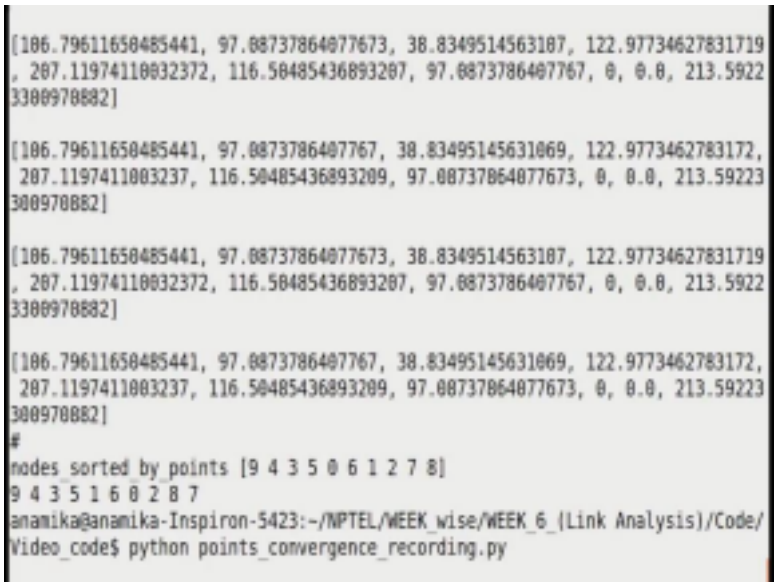
Let us check alright. So, these are the initial values 100 each and after first iteration these are the values that we get. We got 50 points for the first node now, I am pressing enter and the values are changing ok. So, the values are changing.

(Refer Slide Time: 10:41)



Let us the, for first node has 106, only the decimal values are changing. And, let us check the second one, second one is also slightly changing third, fourth I think all the nodes are only slightly changing. So, we can stop at this moment or maybe you can go ahead a little more ok. I am just keeping on pressing enter; it is just changing very slightly. It might take number of iterations to stop. So, maybe we can press hash and stop at this point.

(Refer Slide Time: 11:15)



This is something that I want to tell you. So, this is the first thing that you can see is the ranking that we got by our method ok. And, the second row is the values that we got by the page rank method from networkx. Now, let us compare 9 4 3 5 is correct after that 0 6 1 2 7 8. So, we can we see some changes here.

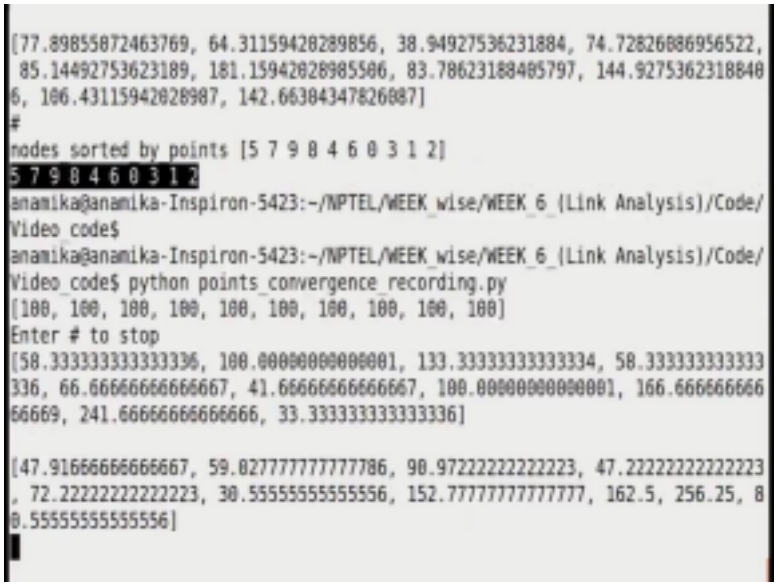
This is what I wanted to show you, why these changes are coming and how we can handle that ok. So, can you imagine what can happen if there is a node which has no out link. So, as we also implemented if a node has no out link, this node does not distribute its points. So, in every iteration, what happens it keeps on changing; it keeps on getting points ok. It keeps on getting point from other nodes and it never distributes its own points. So, what happens is that the point sink happens.

Sink basically all the points are getting accumulated at 1 node or a set of nodes. So, this example I gave for 1 node, it may also happen that in the graph that we generated there are a bunch of nodes which are not distributing any points outside. So, basically there is a link from 1 node to the other and there is a link from second node to the first node, but there is no link outside. So, these 2 nodes will keep accumulating all the points and that is how the ranking that we will get will not be accurate.

So, for that we need to take some measures, we must handle that case. The networkx function is basically handling those cases whereas; our function is not handling those

cases; that is why we are getting these differences in the ranking. Now, we are going to handle these cases. Let me tell you it will not happen in all the cases, because in all the cases your graph might not be having such sinks right. Let us check one more example, if we are getting the same. So, let us check for the convergence first.

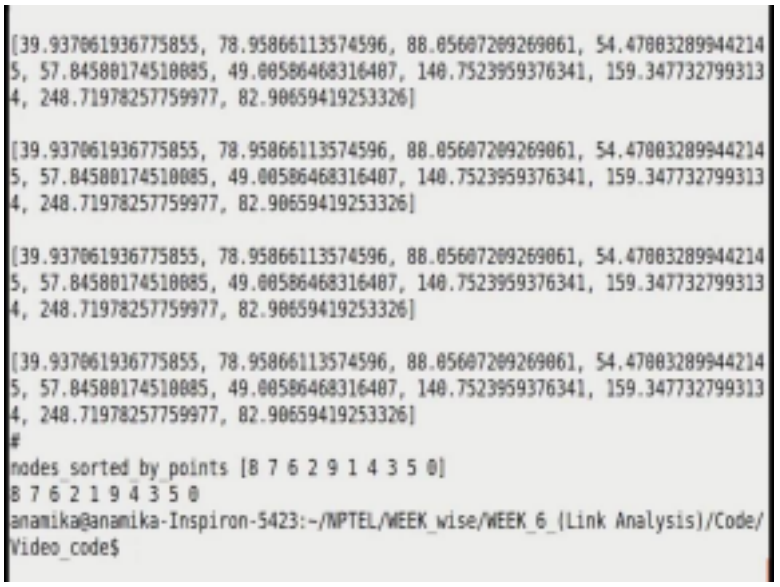
(Refer Slide Time: 13:31)



Again the values are keeping changing ok, now I am pressing enter and it is not going dead basically the values are not at all changing. So, it is completely stopped, there is no change now I am keeping on pressing enter. So, we can just press hash, let us see the ranking that we getting we are getting 5 7 9 8 4 6 0 3 1 2 ok, that is again another thing I want to show. So, this might be a graph where there is no such problem as I just explained you there is no sink.

There is no 1 node or a set of nodes which are keeping on accumulating all the points and they are not distributing that case is not here and that is why we are getting the same thing here. So, our method is accurate just in case there is no sink there in the graph ok. Let me just take another case and I am sure there will be some problem here. I actually want some problem to happen so, that the next video we can cover that because, we are going to handle such cases.

(Refer Slide Time: 14:28)



I am again pressing the enter and it is not going ahead, it is stopped. So, 8 7 6 2 1 you pick 2 1. Yeah, we got the difference nice. So, 2 9 1 and 2 1 9 so, we got a difference right; let me tell you one more thing sometimes there might be a slight you know variation for another reason as well. And, that another reason could be that 2 nodes are having exactly same page rank values right.

So, in that case your method can rank them in a different order and page rank method from networkx for example, can rank them in a different order. So, that sort of difference can happen even if your method is completely working fine and taking care of the sink cases as well. So, that you can just ignore, but as of now we know that our code is not taking care of the sink cases. So, we need to do that ok. So, let us get back to our code.

(Refer Slide Time: 15:30)

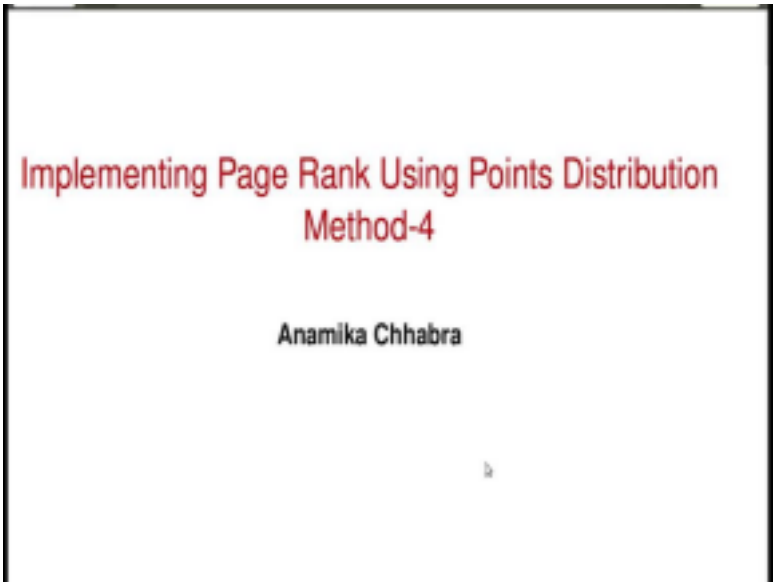
```
67  G = nx.DiGraph()
68  G.add_nodes_from([i for i in range(10)])
69  G = add_edges(G, 0.3)
70
71  # Assign 100 points to each node.
72  points = initialize_points(G)
73  print points
74
75  # Keep distributing points until convergence.
76  G, points = keep_distributing_points(G, points)
77
78  # Get nodes' ranking as per the points accumulated.
79  nodes_sorted_by_points = get_nodes_sorted_by_points(points)
80  print 'nodes_sorted_by_points', nodes_sorted_by_points
81  # Compare the ranks thus obtained with the ranks obtained from the inb
82
83  pr = nx.pagerank(G)
84  pr_sorted = sorted(pr.items(), key = lambda x:x[1], reverse = True)
85  for i in pr_sorted:
86      print i[0],
87
```

And in the next video we will be handling that case.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Link Analysis
Lecture – 83
Implementing Page Rank Using Points Distribution Method – 4

(Refer Slide Time: 00:06)



In the previous video, we finished Implementing Page Rank using Points Distribution Method and it was in most of the cases coming same as the ranking that the networkx a function page rank provides us. However, in some cases the ranking was not similar; the reason for that was that our code was not handling the cases, where one node or a set of nodes have no out links. So, in that case all the points get accumulated in that node or that set of nodes. So, our code has to handle those cases. That is what we will be doing in this video.

Let us get back to our code and see what kinds of measures are required to handle those cases.

(Refer Slide Time: 00:51)

```
34
35
36 def keep_distributing_points(G, points):
37     prev_points = points
38     print 'Enter # to stop'
39     while(1):
40         G, new_points = distribute_points(G, prev_points)
41         print new_points
42
43         char = raw_input()
44         if char == '#':
45             break
46         prev_points = new_points
47
48     return G, new_points
49
50 def get_nodes_sorted_by_points(points):
51     points_array = np.array(points)
52     nodes_sorted_by_points = np.argsort(-points_array)
53     return nodes_sorted_by_points
54
```

(Refer Slide Time: 00:56)

```
67     G = nx.DiGraph()
68     G.add_nodes_from([i for i in range(10)])
69     G = add_edges(G, 0.3)
70
71     # Assign 100 points to each node.
72     points = initialize_points(G)
73     print points
74
75     # Keep distributing points until convergence.
76     G, points = keep_distributing_points(G, points)
77
78     # Get nodes' ranking as per the points accumulated.
79     nodes_sorted_by_points = get_nodes_sorted_by_points(points)
80     print 'nodes_sorted_by_points', nodes_sorted_by_points
81     # Compare the ranks thus obtained with the ranks obtained from the
82
83     pr = nx.pagerank(G)
84     pr_sorted = sorted(pr.items(), key = lambda x:x[1], reverse = True)
85     for i in pr_sorted:
86         print i[0],
87
```

So, this was our main. And here, we were calling this function keep_distributing_points and let us get back to this function and see what it was doing. So, keep distributing functions was keeping on calling this function distribute_points. This distribute_points was basically, 1 iteration that was returning us updated points after that iteration right.

Now, after every iteration we have to make some changes to the points in order to handle the cases of sinks, point sinks. Now, how that is usually done let me explain that to you. So basically, what we do is after every iteration, we gather a fraction of points from every node and the points that we gather, this way we then distribute these points equally

amongst all the nodes right. So basically, for example, after 1 iteration we get some points with all the nodes. And after that to handle the sink cases, we take one more step and, in that step,, we let the node have 80 percent of it is points and we take some 20 percent of the points from that node. And we do that with every node. And that is how we gather some number of points, after that we divide these points that we have gathered this way amongst all the nodes equally.

Now this is the way to handle the problem of sink. You can consider this to be similar to the tax system in any country, where we take some say 30 percent of the salary from every person, if their salary is exceeding some value. So, the amount of money that we gather this way that is distributed for the development of the rest of the people or all the people in general. So, it is the same scenario here as well, the number of points that the node will be all the nodes will be keeping.

And the fraction of points that they will be letting go is basically decided by a parameter, which is usually denoted by s. So, usually we take s = 0.8, which means 80 percent of the nodes points will be with the nodes and 20 percent of the nodes points will be let go by all the nodes right. So, let us increment that thing here. As you can see in this function keep_distributing_points, we call this function and that is how we got the new points.

(Refer Slide Time: 03:44)

```
31                  new_points[each[1]] += share
32
33      return G, new_points
34
35
36  def keep_distributing_points(G, points):
37      prev_points = points
38      print 'Enter # to stop'
39      while(1):
40          G, new_points = distribute_points(G, prev_points)
41          print new_points
42
43          new_points = handle_points_sink(G, new_points)
44
45          char = raw_input()
46          if char == '#':
47              break
48          prev_points = new_points
49
50      return G, new_points
51
```

Now, after we get the new points, let us take one more step to handle um the sinks let me

for that purpose create a function. So, I will write new points, is equal to handle points sink let us call this function. So, let us pass the graph and the new points here ok. So, we have to create this function, now which will be doing, whatever I just explained that is it will be taking 20 percent of the points from every node and whatever we gather this way we will be distributed to all the nodes equally.

(Refer Slide Time: 04:16)

```
23
24      for i in G.nodes():
25          out = G.out_edges(i)
26          if len(out) == 0:
27              new_points[i] += prev_points[i]
28          else:
29              share = (float)(prev_points[i])/len(out)
30              for each in out:
31                  new_points[each[1]] += share
32
33      return G, new_points
34
35  def handle_points_sink(G, points):
36      for i in range(len(points)):
37          points[i] = (float)(points[i])*0.8
38
39      n = G.number_of_nodes()
40      extra = ((float)(n)*100*0.2)/n
41      for i in range(len(points)):
42          points[i] += extra
43      return points
```

So, let us create this function ok. So, as a first step every nodes point will be reduced right, every nodes point will be reduced to 80 percent of what they were. So, let me start look here for i in range(len(points)) we can generalize it to may be points and that is what it will be returning. So, for in range(len(points)), we must reduce the points for every node. So, we will write points[i] = points[i] * 0.8. So, they become 80 percent or what they were earlier let me do the casting ok.

So now, every node has reduced points reduced to 80 percent, now what should be done with the remaining 20 percent of the point that we have gathered from every node? Now if you know that what are the total number of points that the graph had initially: you can understand that then the total number of points never change. They are only sorry; they are only distributed amongst each other or the nodes will keep giving the points to each other, but the total number of points never changed ok.

So, we are going to use that fact to know, what are the points that we have gathered? So, if every nodes point has become 80 percent, we have gathered 20 percent of the total

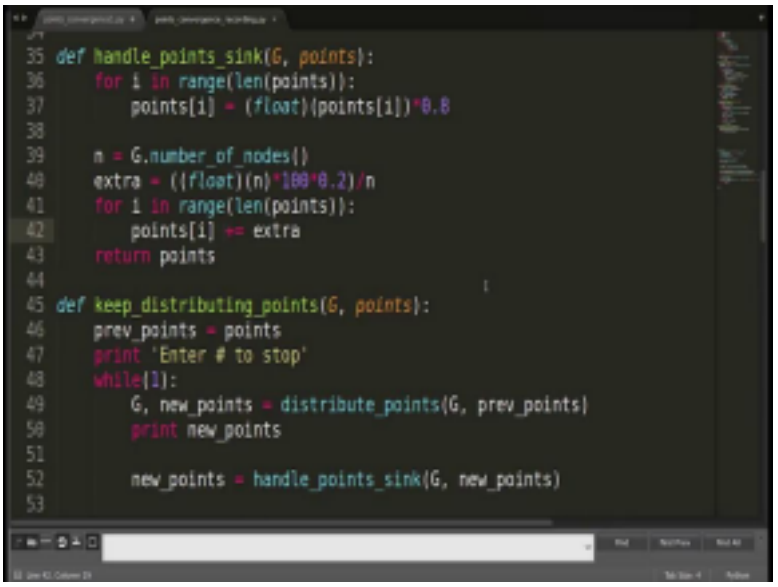
number of points that the graph had ok. Now what is the total number of points that had? We had assigned 100 points to every node ok. Let us see the total number of nodes, I will just show that here. So, I will write g.number_of_nodes ok. So, I got n here what will be the total number of points in the graph that will be n * 100, because we assigned 100 points.

Now all of this these total points 20 percent is what we have now so that we can distribute to every node. So, I will write into 0.2. So, this is what we have ok. Now how many points to we have to distribute to every node? We must distribute these points equally ok. So, we will divide by n ok, so that we get an equal share for every node that will be the casting here ok. Let us call it extra now because, this is the extra points that every node will be having may be, we can took this as well ok.

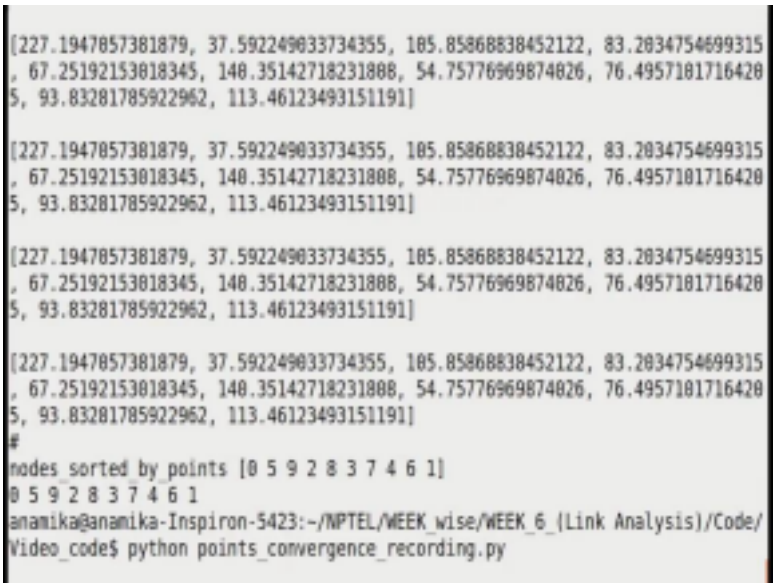
So, these are the extra points that have to be assigned to every node apart from the point that they already have. So again, we will start a loop. I will write for i in range(len(points)) again ok. So, points of every node will become plus equal to extra ok. So, every node is going to get extra points I think we have done. So, just return the points these are the new points ok. So, we passed old points to this function handle_points_sink and it will return the updated points, which will go back to this function keep distributing points here ok.

So, new points have changed, this is what we passed, and this is what we got ok. So, new points have changed since, we had not change the name. So, I do not think we have to change any other thing in the function.

(Refer Slide Time: 08:09)



(Refer Slide Time: 08:14)



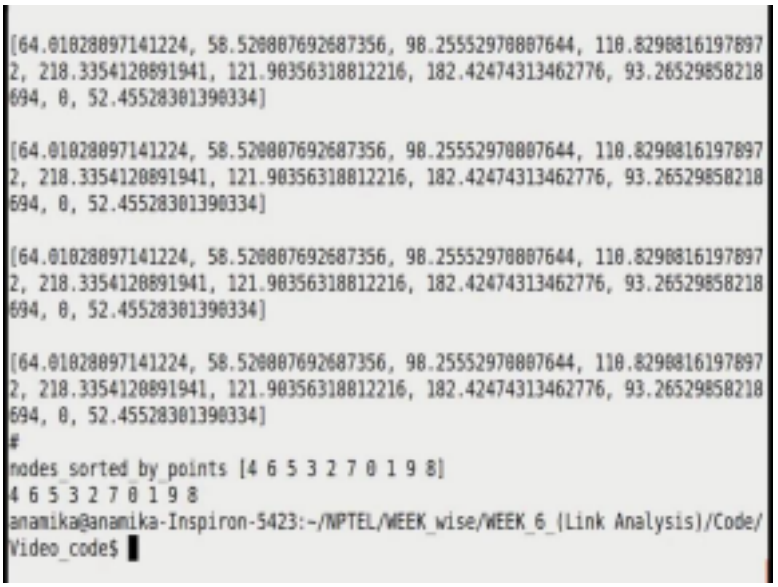
Let see how our program works. So, let us execute, so these are the initial values, and these are this is how the values are changing, I am just keeping on pressing enter and now I will press # ok. So, these are the values that we are getting 0592837. So, they are exactly in matching. So, we are handling the sink cases. So, the values are exactly matching.

Let me tell you one more thing sometimes there might be few differences with the with the ranking that you are getting from the networkx function as I told you, I guess one of the reasons could be 2 of a nodes are having the same page rank another reason could be

you have not let it converge, it is going on and on and it is taking a lot of iterations to reach an exact convergence point.

And you are just stopping somewhere in the middle. So, that might also lead to a slight deviation from the ranking that you are getting from networkx. Let me check one more example here ok.

(Refer Slide Time: 09:21)



So, this is what we are getting let me stop here. 4653270198 here; so, we are getting the same values because, we are handing the case.

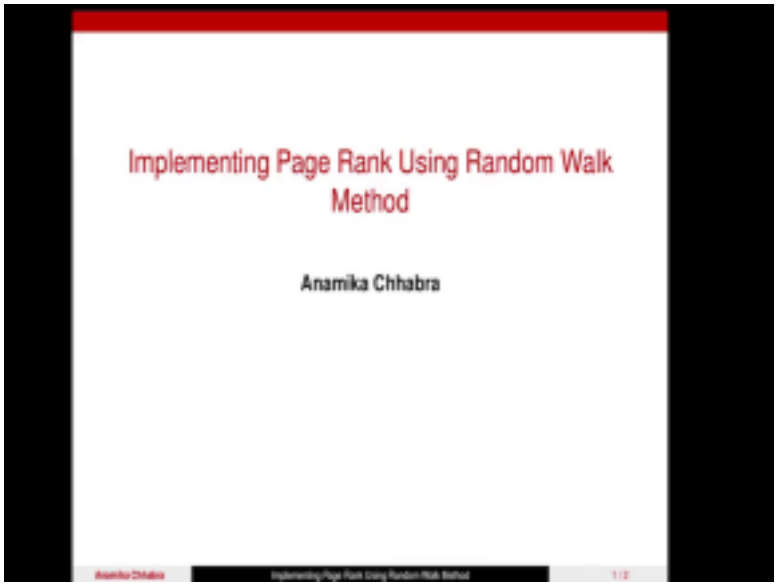
So, in case you get the slight deviations also you need do not worry because, you have understood the whole concept; how basically points distribution is similar to getting a page rank because, apart from the mathematical complexities this is what basically is the main idea behind getting the nodes rank based on the page rank values.

So, this was about the implementation of page rank using point distribution method. So, we are going to implement random walk method in the next videos.

Social Networks
Prof. S.R.S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Link Analysis
Lecture – 84
Implementing PageRank Using Random Walk Method – 1

(Refer Slide Time: 00:05)



Hey everyone, in the previous video we implemented page rank using points distribution method. In this video we are going to Implement Page Rank Using Random Walk Method. Now what exactly is random walk method? So, basically, we are going to choose 1 node from the network uniformly at random. And, we will look at its neighbors; we will choose one neighbor out of them uniformly at random. And then we will look at the neighbors of that node, and we will keep doing this. And that is how we will randomly walk on the network that precisely what random walk is.

Now, there are two things that we are going to do along with this walk. And that is first thing that we will do is as and when we choose a node, we will increment its points by 1. Since we used points in the previous method as well which are different sort of points, let us call these points as random walk points just to differentiate things.

So, as I was saying as and when we reach a node, we increment its random walk points by 1 ok. That is one thing that we do, second thing is while we are randomly walking on the network it might so happen that we reach a node which has no out links, which has no neighbors where we can go right. So, in that case what we do is we out of all the nodes in the network we choose one uniformly at random and we resume ours our random walk from that node.

Now this is precisely called teleportation, which is a frequently used term when you talk about page rank and random walks. So, teleportation is basically choosing a node uniformly at random from the network and starting our basically resuming our random walk from that point.

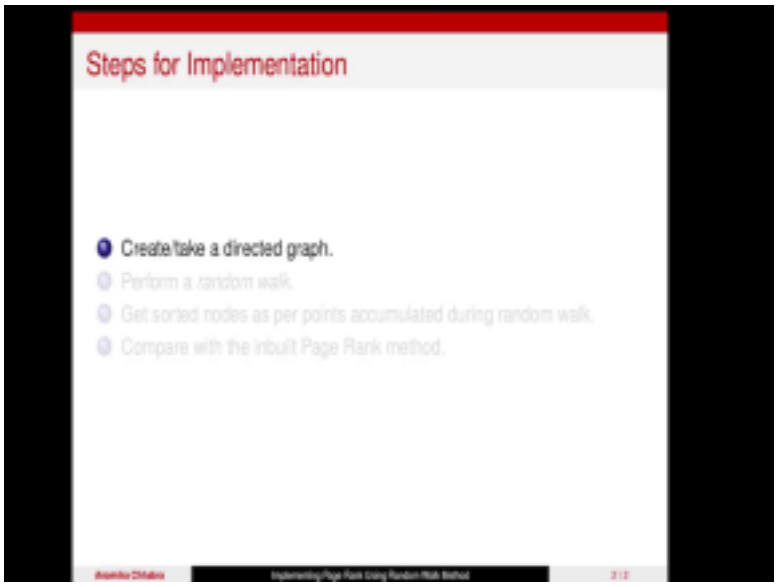
Now, what is the basic idea behind this concept? How is it I mean it is, it looks counterintuitive that we are trying to implement page rank using random walk; it sort of does not make sense intuitively. So, let me tell you the basic idea that is used here. As we randomly walk on the network, a node which has a number of huge number of in links. There is a high probability that we will reach at this node more often right. And anytime we reach a node we are incrementing its random walk points.

So, the number of times we reach a node we are keeping a track of that. And a node which has high number of in links we will be reaching that node quite often. On contrary if we have a few nodes which have very a smaller number of in links, we will be reaching them very less often and that is what is recorded by the number of random walk points. So, that is a basic idea because in page rank also the idea is same.

If lot of nodes point to you then you are in important ok. And another thing that it captures is if a lot of important nodes points to you then you are important. So, in the random walk also that thing will be captured that if a lot of nodes, which are reached quiet often and they are pointing to you, you will be reached quiet often. And, the same thing is captured by the count that we are keeping random walk points that we are keeping. So and based on those points we are going to rank the nodes; which will be precisely same as a ranking that we get from the page rank technique. So, let us see the

steps that we are going to implement ok.

(Refer Slide Time: 03:54)



As in the previous case we are going to take a directed graph. So, we can either take it using some generator from network x function or we can create it ourselves. So, we created in the previous video we are just going to make use of that code again.

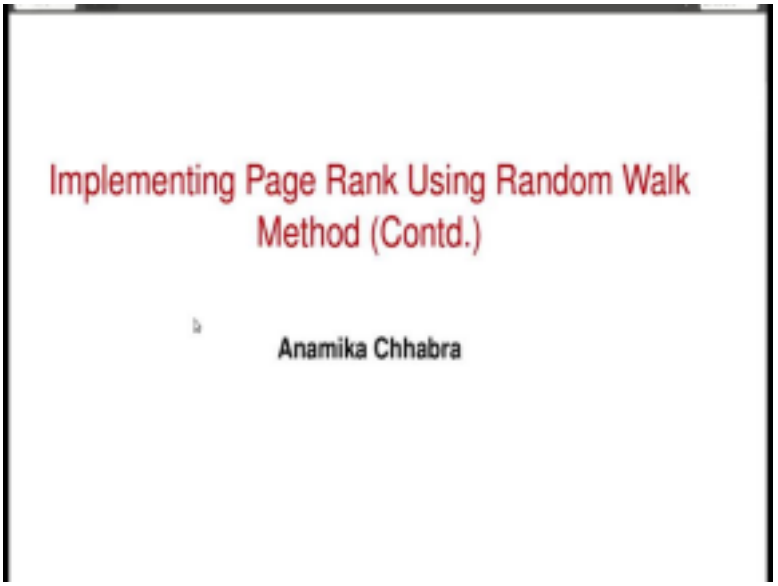
Second step that must be implemented is the main step which is performing a random walk. As I just explained to you, we are going to implement those steps. After performing a random walk, we are going to have count of the number of times we reached a node that is the random walk points. So, we are going to rank the node based on the random walk points that we obtained in the previous step.

In the last step as we did in the previous video as well. We are going to compare a results, with the results that we obtained from the inbuilt page rank method from networkx. So, these are the sequence of steps that we are going to follow; first, third and fourth steps are the same as in the previous video. So, the main thing that has to be implemented is the second one that is the random walk which we are going to in the next video.

Social Networks
Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

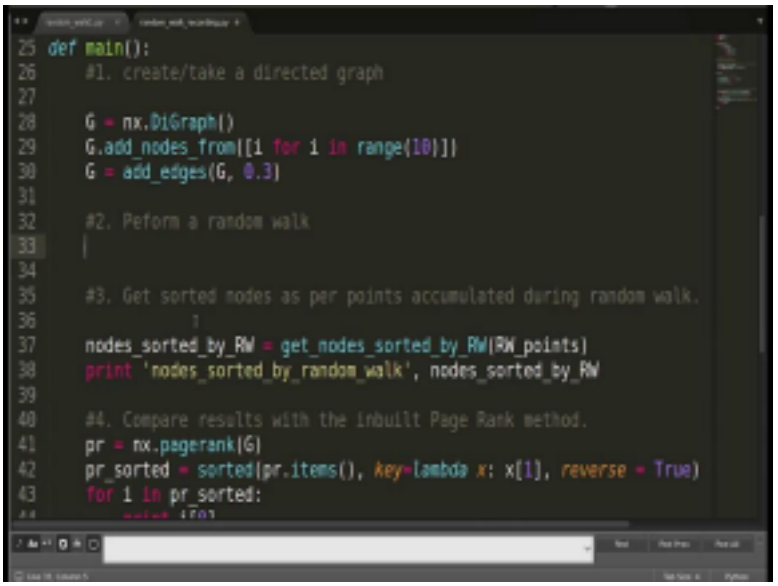
Link Analysis
Lecture – 85
Implementing Page Rank Using Random Walk Method – 2

(Refer Slide Time: 00:07)



In this video we are going to Implement Page Rank Using Random Walk. So, let us start the code.

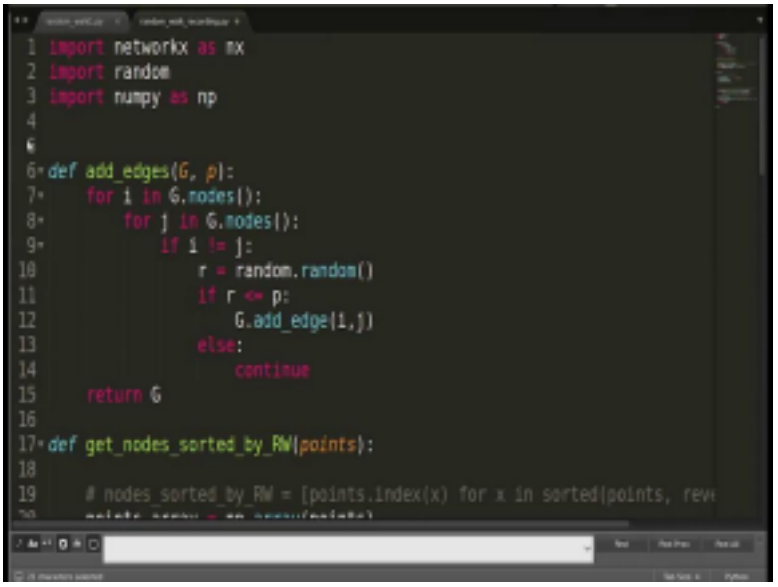
(Refer Slide Time: 00:12)



As I told you, in the previous video there are 4 steps which are to be implemented. And out of those 4 steps first third and forth steps are the same that we implemented in our previous video on implementation of page rank using point distribution method. So, we are going to reuse that code and there is no point of writing the code again.

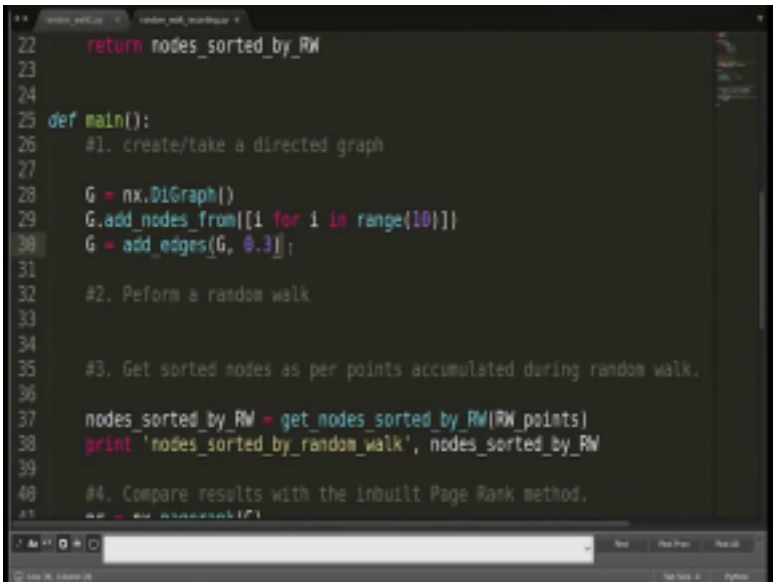
So, I have copied pasted the code which is required over here in this file. Let me show you, we will start from the first step. First step is to take a directed graph. So, as we had created the graph. How had we created? We had taken a an object of nx.DiGraph. So, we created an empty directed graph here and after that we add nodes to it, we added 10 nodes to this graph. And after that we added edges to the node to the graph using this function add_edges.

(Refer Slide Time: 01:07)



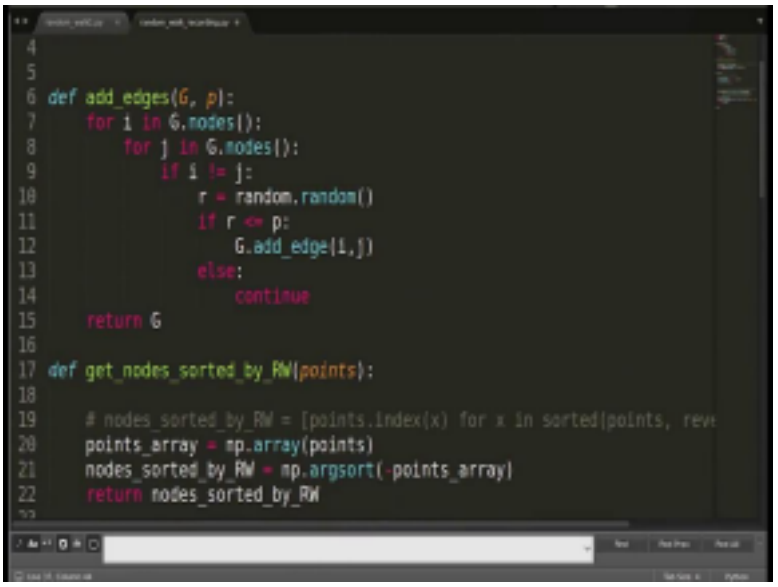
So, let us go back here. These add edges function I have copied as it is from the previous code. So, here just to brief about we are tossing coin for every edge. And if the coin turns out to be head, we are adding that edge otherwise we are not keeping that edge. So, that is how we are randomly assigning edges to the graph.

(Refer Slide Time: 01:35)



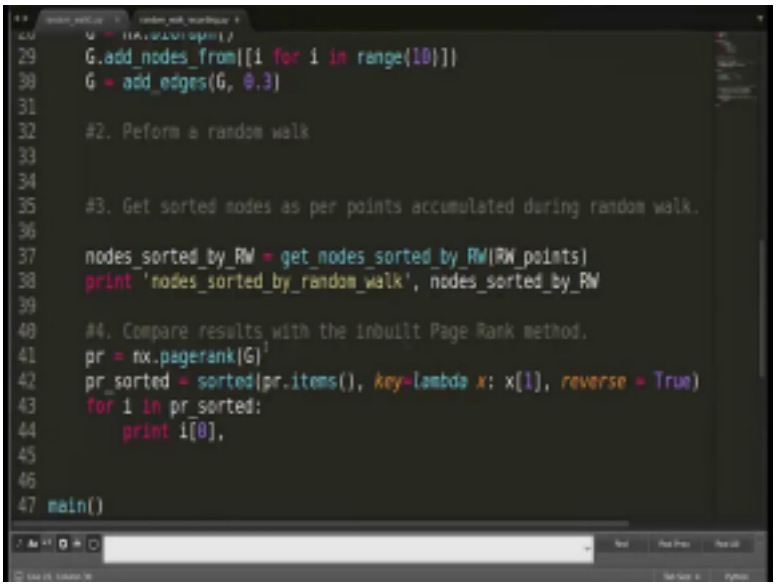
So, after this our graph is ready and we can perform a random walk on this. So, that is a second step performing a random walk which is to be done, we will just do that. After forming the random walk we will be having random walk points. And then we have to rank the node based on random walk points which is what we have done in the previous video. I have just pasted the code; I have just renamed this function get node sorted by random walk. I just renamed it here that is all, the functionalities entirely the same.

(Refer Slide Time: 02:08)



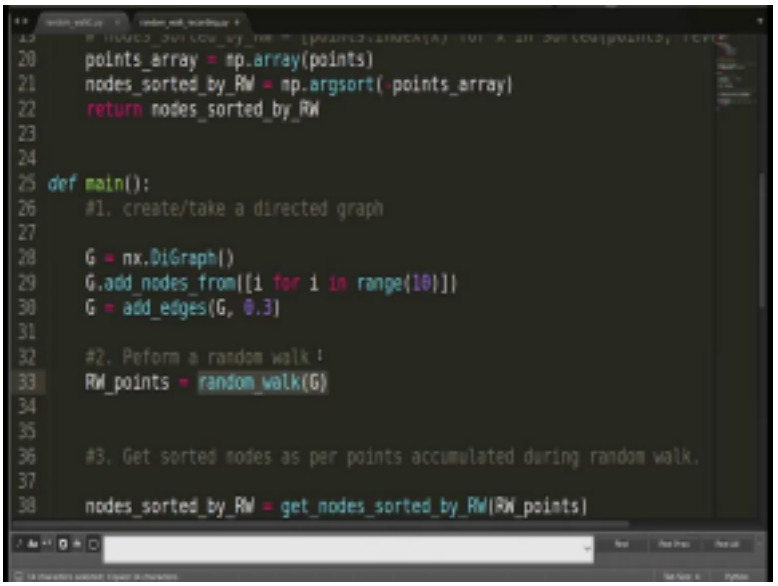
So, this function I have added as it is. So, that part is done.

(Refer Slide Time: 02:13)



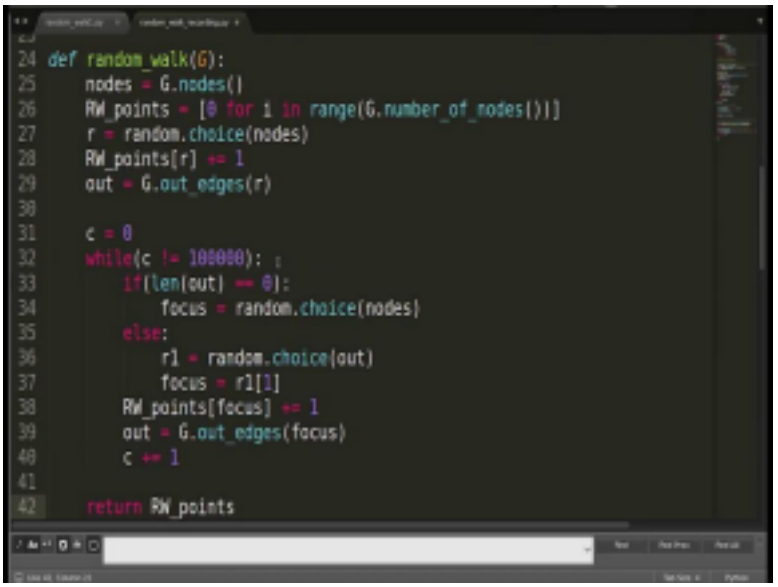
And finally, once we are done, we use the page rank method from network x. And we compare the results, our results with the functions results; again, this is entirely the same code ok. So, what has to be done is the second step which is performing a random walk, now let us do that. So, this function is going to we are going to create a function random walk and that function will return random walk points ok.

(Refer Slide Time: 02:43)



So, let us write that: random walk points is equal to random walk. So, we are going to create this function random walk, we will pass the graph there ok. So, let us create this function.

(Refer Slide Time: 02:59)



Now, as I told you what do we mean by random walk? We will randomly choose, 1 node from all the nodes that, is the first step ok how do we do that? Let us create let us get a list of nodes firstly. So, let us write node = G.nodes because, we have chosen one randomly we will choose out of these out of these list. And as and when we visit a node, we increment its counter, increment its random walk points. So, it will be nice to create a list which will be returned by this function, random walk points is equal to we have to initialize it to 0. So, we can write 0 for i in range G.number_of_nodes. So, random walk points initialize to 0.

Now, we have this list of nodes and we have to choose one out of them out of these nodes randomly. So, we can do r is equal to random dot. We have to choose 1 node randomly out of these lists; for that we can use a function random dot choice, random dot choice out of nodes ok. So, we got 1 node uniformly at random. Since, we are have visited it we must increment its counter its point. So, we will write RW points for this node r, must be incremented by 1.

After that we have to look at the nodes which are neighbors to this node ok. We have to choose one neighbor uniformly at random. So, let us first get a list of the neighbors of this node are. How do we get that? For a directed graph we have function G dot out edges ok. G dot out edges for r, out will contain a list of all the edges, out edges from r ok. And after that we have to choose one of the, one of the neighbors uniformly at

random and we have to repeat the same process. So, basically what are we are doing, we have to keep repeating it. How many times we should do, as much as possible ok.

So, let us create a counter for that. And we can start a loop because, we have to keep repeating things, we can take a very big number probably whatever as much as you can handle. I have taken this big number. So, we will visit these many nodes. So, we will basically have these many iterations of a random walk from one node to the other. So, we can keep any number here. In the basic idea is that we should be able to visit all the nodes; we there should not be nodes which we have not visited because if you do not visited node the counter for them will be 0. And, hence we will not be able to rank the nodes properly.

So, the idea is that we should be able to visit all the nodes. And that too multiple times so, that we are able to capture the frequency of visiting in our counts; in our random walk points ok. So, what do we have to keep repeating? We are started the loop. What do we have to keep repeating? We had reached r and these are the neighbors of r, we have to choose one uniformly at random out of them ok. So, that is what we will do. Before that what if this out is 0, the length of out is 0 that is the node which we have reached has no out links that may also happen right. And, I told you in that cases what we do? In that case we do teleportation, that we that is we choose one node uniformly at random from all the nodes.

So, let us check that if length of out is equal to 0, what am I doing? If the length of out is equal to 0. What we do? We have to choose one uniformly random let us create a new variable for that to keep the node which is currently under focus ok. This focus will contain the node which is currently under focus, which is currently being visited basically. So, we have to choose one node uniformly at random from all the nodes we will write random dot choice nodes ok. This is the list which we already created. In case the node which we have reached at has number of neighbors, what do we have to do? We have to choose one neighbor out of them.

So, maybe we can have another random variable here, we can write r1 dot. I am sorry random.choice. Now this choice has to be made out of what? This choice has to be made out of out. Because out is what is containing all the edges and we will choose one edge

out of them. Now, what is going to be the focus? The focus is not going to be the edge. The focus is going to be the node which is on the other side. How do we get the node, which is on the other side, using r1[1] because r1[0] , r1[1] is basically the edge ok yeah exactly?

So, we got the focus where focus is the current node where we are currently. Since we have reached at this node. We have to implement the random walk points for this node. So, we will write random walk points for focus plus is equal to 1. Now again we have to get the list of its neighbors. So, we will write out is equal to we can copy paste this, I am just set r will be replaced by focus; because focus, focus is what is a current node.

Now, this is one iteration; what is iteration having? This iteration consists of looking at the neighbors choosing one out of them randomly and then it be and then repeating the same process. That is what this while looping is doing. Let us implement the count c that is the number of times this while loop will continue ok. I think we are done. So, what we have to return? This function should return random walk points right. So, let us return this; I think we are done. Let us go down to the main, in the main also we have implemented everything.

(Refer Slide Time: 10:05)

```
    #1. create/take a directed graph

    G = nx.DiGraph()
    G.add_nodes_from([i for i in range(10)])
    G = add_edges(G, 0.3)

    #2. Peform a random walk
    RW_points = random_walk(G)

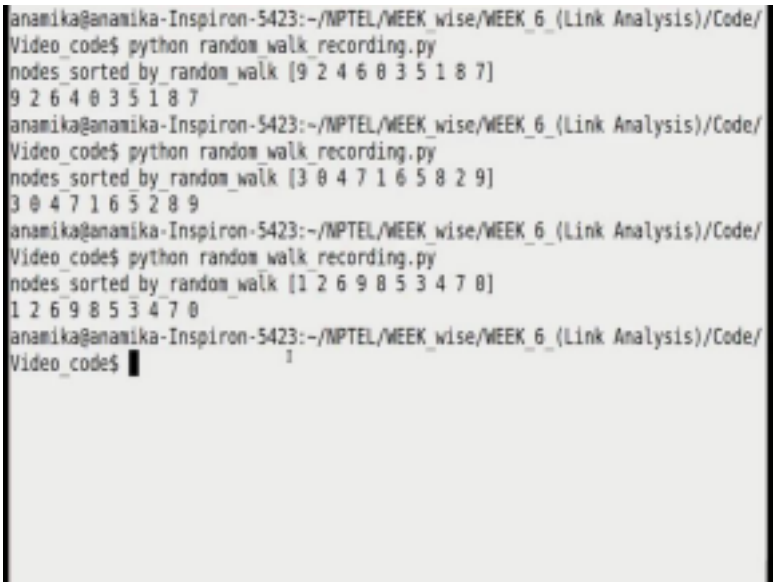
    #3. Get sorted nodes as per points accumulated during random walk.

    nodes_sorted_by_RW = get_nodes_sorted_by_RW(RW_points)
    print 'nodes_sorted_by_random_walk', nodes_sorted_by_RW

    #4. Compare results with the inbuilt Page Rank method.
    pr = nx.pagerank(G)
    pr_sorted = sorted(pr.items(), key=lambda x: x[1], reverse = True)
    for i in pr_sorted:
        print i[0],
```

We only have to write this instruction rest all are the same from the previous code. I think we are good to go, let us check whether this code works.

(Refer Slide Time: 10:19)



So, let us so, as you can see the outputs 9, 2, 4, 6 and here 9, 2, 6, 4 and rest of the values are matching. So, there can be two reasons for the miss match. One could be the rank of 4 and 6 might be same, 4 and 6 here and 6, 4 and 4 here. So, they can give different ordering. Second reason could be that we needed more iteration so, that we were able to visit all the nodes properly and we could achieve the effect. Because random walk is basically based on probability right, you randomly choose the next node. It might so, happen that you are ending up reaching one node, more than the other node whereas, they both have the same number of in links right.

So, that may happen, let us check another example. Again, here a everything is same except 8 2 and 2 8 over here; we can get one more. So, in this case used as you can see it is precisely the same, everything is matching. So, it might happen because it is based on probability and it is never possible that there are 2 nodes which have the same configuration in terms of the number of in links. And, we are reaching these two nodes precisely the same number of times. So, that sort of difference is going to happen. So, this was about the implementation of page rank using random walk technique.

Social Networks

Prof. S. R. S. Iyengar
Department of Computer Science
Indian Institute of Technology, Ropar

Lecture – 86
Degree Rank versus Page Rank

(Refer Slide Time: 00:05)



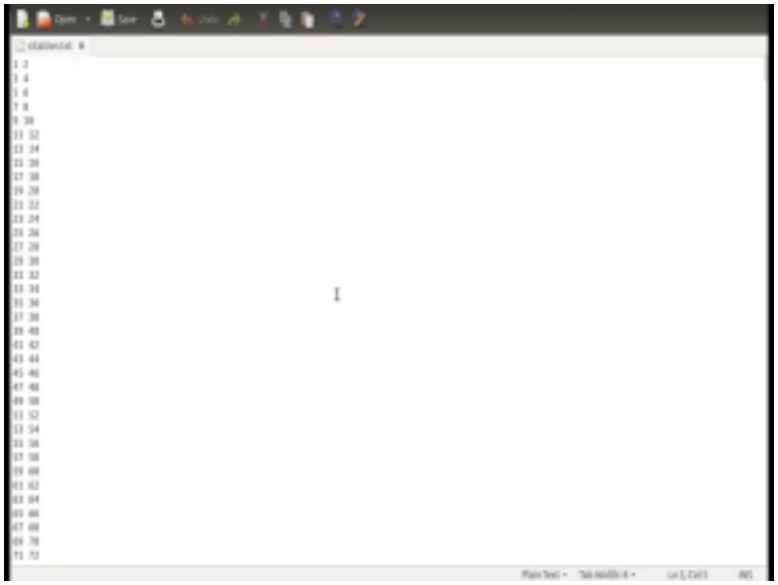
Hey everyone, in this video we going to see the relationship between Degree Rank and Page Rank that is we are going to see whether they correlate with each other or not? We are going to see whether the nodes which have less degree do they have less page rank and vice versa. So, in order to check this, we are going to use citation network which is a directed graph I already have downloaded it from snap repository, so I am going to use that. So, let us start the implementation.

(Refer Slide Time: 00:34)



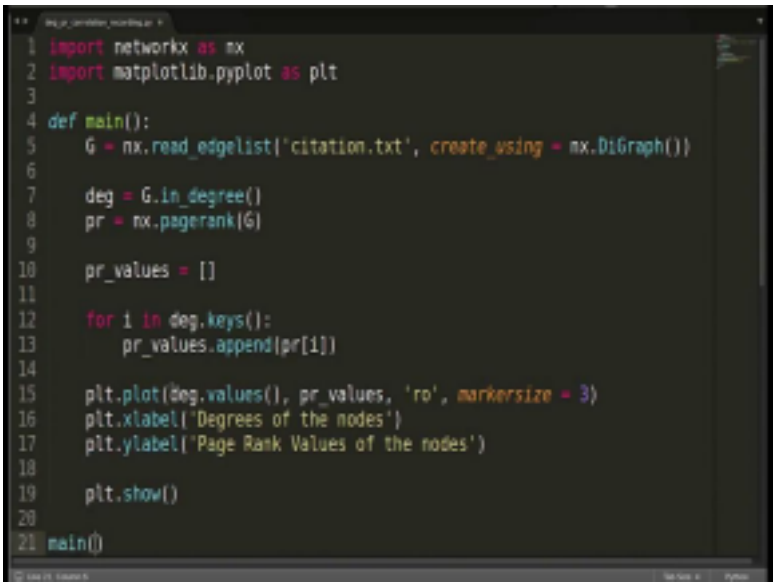
Let me show you the citation data set how it looks like?

(Refer Slide Time: 00:38)



So, this is the citation data set as you can see it is in the form of edge list. Every row contains an edge these are the IDs of the research papers. So, if you take an edge it indicates that the first paper is citing the second paper. So, the paper which there is ID 17 is citing in the paper with the ID 18. So, this is a sort of information that we have.

(Refer Slide Time: 01:04)



```
import networkx as nx
import matplotlib.pyplot as plt

def main():
    G = nx.read_edgelist('citation.txt', create_using = nx.DiGraph())

    deg = G.in_degree()
    pr = nx.pagerank(G)

    pr_values = []

    for i in deg.keys():
        pr_values.append(pr[i])

    plt.plot(deg.values(), pr_values, 'ro', markersize = 3)
    plt.xlabel('Degrees of the nodes')
    plt.ylabel('Page Rank Values of the nodes')

    plt.show()

main()
```

Let us implement this now. Since the network is in the form of a edge list we are going to use the function read edge list in order to read it into a graph for object. First, let me import networkx we are going to plot there is a so let me import matplotlib ok.

So, let us start the main function we have to read the network. So, G = nx.read_edgelist, the network name is citation.txt. Since we have to create a directed graph, we will write create using is equal to nx.DiGraph.

Now since this is a directed graph, we are going to keep a track of the in degrees of the nodes that is basically what we will compare with the page rank. So, we have to check whether a node which has more in degree that is a greater number of in links. Does it have more page rank is well or not and vice versa.

So, let us first get all the in degrees we can use a function G.in_degree for that purpose. Now what this function return is a dictionary. So, this is going to be a dictionary where the keys are going to the nodes and the values are going to be the in_degree values of these nodes.

Similarly, in order to compute the pr we are going to use the inbuilt functions from networkx. We will write the page rank = nx.pagerank(G), again page rank function will return a dictionary where the keys are the nodes and the values are the page rank values of these nodes.

Since these two are two different dictionaries and we must plot them. We basically we are going to keep the in_degree values on the x axis and the corresponding page rank values on the y axis. Since there is a two different dictionaries let us keep one of them in the order of the other.

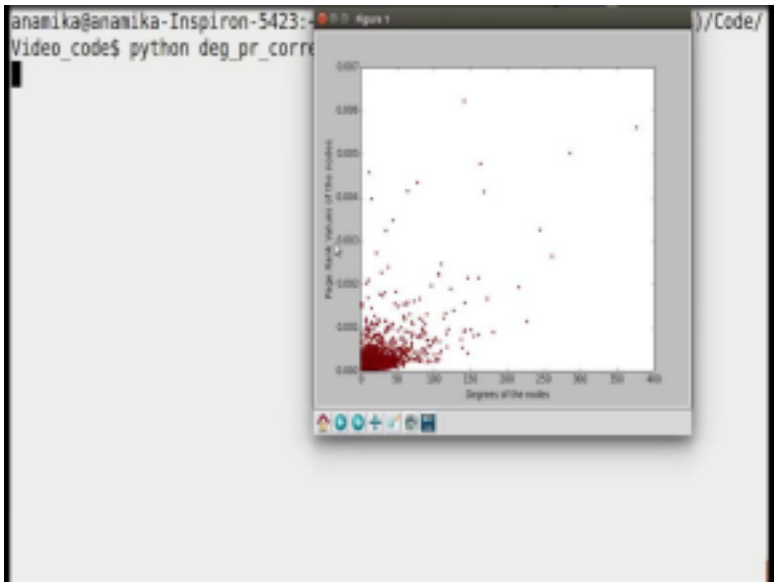
So, I am going to create a list to keep the page rank values in the same order in which degree values are coming. So, I am creating this list page rank values is equal to a list. So, I am going to create this list which will have the page rank values of the nodes in the same order in which they are pairing in this degree dictionary ok.

So, for that I am starting a loop in deg.keys. So, deg.keys will give the keys in the sequence, then we will append the page rank values pr_values.append. What do we have to append? We have to append the page rank value of the corresponding node i ok. So, we will write pr[i] ok, so we got the both the things in the same sequence and then we can plot them.

So, I will write plt.plot on the x axis we have in degrees which are available in this dictionary. So, we will write deg.values corresponding to these degree values what are the page rank values is what is available in pr_values list that we just created. Let us display it in red circles; we can also change the marker size ok.

Let us also give the labels, degrees of the nodes and we can also give the y label on y axis we have page ranks, page rank values ok. After that we have to show we will write plt.show sorry ok. Then we will call this main function they should work. Let us see how it works?

(Refer Slide Time: 05:55)



As you can see in the plot degree and page rank values are not correlated with each other. Because if they if they had been correlated, they would have been linear sort of plot over here which is not there. And, you can see that there are few nodes which have high in degree; however, very less page rank.

For example, if you take this node it has high in degree, but if you look at its page rank which is less. In the context of citation network, it indicates that this is a research paper which is being cited by several other papers. However, those papers are not so important right that is why this paper got less page rank.

Similarly, if you look at the other side you have a few nodes which have very less in degree; however, extremely high page rank. Now in the context of citation network, it indicates that these are the research papers which are being cited by very fewer other papers. However, those papers are important themselves that is those papers are being cited by a lot of other papers, or those papers are being cited by a lot of important papers. So, the conclusion is that in degree and page rank values are not correlated with each other.